

RTL Design Sherpa

RTL AMBA APB5 Module Reference — Rev 0.91

October 2026

Table of Contents

1 APB5 (Advanced Peripheral Bus - AMBA 5) Modules....13
1.1 Overview....13
1.1.1 AMBA4 vs AMBA5 Comparison....13
1.1.2 Module Categories....14
1.1.3 Key Features....15
1.2 Functional Description....16
1.2.1 APB5 Transfer Phases....16
1.2.2 APB5 Signal Descriptions....16
1.2.3 Optional Parity Signals....17
1.3 Usage Examples....19
1.3.1 Using APB5 Master....19
1.3.2 Using APB5 Slave....20
1.4 Design Notes....22
1.4.1 Command/Response Architecture....22
1.4.2 Migration from APB4....22
1.5 Related Modules....22
1.6 Testing....22
1.7 References....23
1.7.1 Specifications....23
1.7.2 Papers and textbooks....23
1.7.3 Source Code....24
1.8 Navigation....24

2 APB5 Master....25
2.1 Overview....25
2.1.1 Key Features....25
2.2 Parameters....25
2.2.1 Derived Parameters (do not override)....26
2.3 Ports....27
2.3.1 Clock and Reset....27
2.3.2 APB5 Master Interface....27
2.3.3 Parity Signals (Optional)....28
2.3.4 Command Interface....29
2.3.5 Response Interface....30
2.3.6 Status Outputs....30
2.4 Functional Description....31
2.4.1 Block diagram....31
2.4.2 APB5 Extensions....32
2.5 Timing Characteristics....32
2.5.1 Basic Write Transaction....32
2.5.2 Basic Read Transaction....33
2.5.3 Wake-up Signal Handling....33
2.6 Usage Examples....33
2.7 Design Notes....35
2.7.1 APB5 vs APB4 Differences....35
2.7.2 FIFO Sizing....35
2.7.3 Parity Implementation....35

2.8 Related Modules....37
2.9 Testing....38
2.10 References....38
2.11 Navigation....39
3 APB5 Master (Clock-Gated)....40
3.1 Overview....40
3.1.1 Key Features....40
3.2 Parameters....40
3.2.1 Derived Parameters (do not override)....41
3.3 Ports....41
3.3.1 Clock and Reset....41
3.3.2 Clock Gating Configuration....42
3.3.3 Clock Gating Status....42
3.3.4 APB5 Master Interface....42
3.3.5 Parity Signals (Optional)....43
3.3.6 Command Interface....44
3.3.7 Response Interface....45
3.3.8 Status Outputs....45
3.4 Functional Description....46
3.4.1 Block diagram....46
3.4.2 Clock Gating Behavior....46
3.5 Timing Characteristics....47
3.6 Usage Examples....48
3.7 Design Notes....48

3.7.1 Power Savings....49
3.8 Related Modules....49
3.9 Testing....49
3.10 References....50
3.11 Navigation....50
4 APB5 Master Stub....51
4.1 Overview....51
4.1.1 Key Features....51
4.2 Parameters....51
4.2.1 Derived Parameters (do not override)....52
4.3 Ports....53
4.3.1 Clock and Reset....53
4.3.2 APB5 Master Interface....53
4.3.3 Parity Signals (Optional)....54
4.3.4 Command Packet Interface....55
4.3.5 Response Packet Interface....55
4.3.6 Status Outputs....55
4.4 Functional Description....56
4.4.1 Block diagram....56
4.4.2 Packet Formats....57
4.4.3 Transaction Flow....58
4.5 Timing Characteristics....58
4.6 Usage Examples....59
4.7 Design Notes....60

4.7.1 First/Last Markers....60
4.7.2 Internal Architecture....60
4.8 Related Modules....61
4.9 Testing....61
4.10 References....61
4.11 Navigation....61
5 APB5 Monitor....62
5.1 Overview....62
5.1.1 Key Features....62
5.1.2 Not Implemented....62
5.2 Parameters....62
5.2.1 Derived Parameters (do not override)....63
5.3 Ports....64
5.3.1 Clock and Reset....64
5.3.2 Command Interface Monitoring....64
5.3.3 Response Interface Monitoring....65
5.3.4 APB5 Extension Monitoring....65
5.3.5 Configuration Inputs....65
5.3.6 Address-Range Checker Configuration....66
5.3.7 Monitor Bus Output....67
5.3.8 Status Outputs....68
5.4 Functional Description....69
5.4.1 Block diagram....69
5.4.2 Monitor Packet Format....70

5.4.3 Packet Types....70
5.4.4 Event Edge Detection....71
5.4.5 APB5-Specific Event Codes....71
5.4.6 Event Priority....72
5.4.7 Wake-up Monitoring....73
5.5 Timing Characteristics....73
5.6 Usage Examples....74
5.7 Design Notes....75
5.7.1 Transaction Table....75
5.7.2 Internal FIFOs....76
5.7.3 Configuration Guidelines....76
5.8 Related Modules....76
5.9 Testing....76
5.10 References....76
5.11 Navigation....77
6 APB5 Slave....78
6.1 Overview....78
6.1.1 Key Features....78
6.2 Parameters....78
6.2.1 Derived Parameters (do not override)....79
6.3 Ports....79
6.3.1 Clock and Reset....79
6.3.2 APB5 Slave Interface....80
6.3.3 Parity Signals (Optional)....81

6.3.4 Command Interface (to backend)....82
6.3.5 Response Interface (from backend)....82
6.3.6 Status Outputs....83
6.4 Functional Description....84
6.4.1 Block diagram....84
6.4.2 APB5 Slave Protocol....84
6.5 Timing Characteristics....85
6.5.1 Write Transaction with Wait States....85
6.5.2 Read Transaction....86
6.5.3 Wake-up Signaling....86
6.6 Usage Examples....86
6.7 Design Notes....88
6.7.1 Backpressure Handling....88
6.7.2 User Signal Propagation....88
6.7.3 Wake-up Input....88
6.7.4 Parity Implementation....88
6.8 Related Modules....90
6.9 Testing....90
6.10 References....91
6.11 Navigation....91
7 APB5 Slave CDC....92
7.1 Overview....92
7.1.1 Key Features....92
7.2 Parameters....92

7.2.1 Derived Parameters (do not override) 93
7.3 Ports 94
7.3.1 Clock and Reset 94
7.3.2 APB5 Slave Interface 94
7.3.3 Backend Interface 94
7.4 Functional Description 95
7.4.1 Architecture 95
7.4.2 CDC FIFO Pointer Encoding 96
7.4.3 Reset Synchronization 97
7.5 Timing Characteristics 98
7.5.1 Latency 98
7.5.2 Timing Considerations 98
7.6 Usage Examples 99
7.7 Design Notes 100
7.7.1 FIFO Depth Sizing 100
7.8 Related Modules 100
7.9 Testing 100
7.10 References 100
7.11 Navigation 101
8 APB5 Slave CDC (Clock-Gated) 102
8.1 Overview 102
8.1.1 Key Features 102
8.2 Parameters 102
8.2.1 Derived Parameters (do not override) 103

8.3 Ports....104
8.3.1 Clock and Reset....104
8.3.2 Clock Gating Configuration....104
8.3.3 APB5 Slave Interface....104
8.3.4 Backend Interface....104
8.3.5 Status Outputs....104
8.4 Functional Description....105
8.4.1 Architecture....105
8.4.2 Wake-up Logic....106
8.5 Timing Characteristics....106
8.6 Usage Examples....106
8.7 Design Notes....107
8.7.1 Power and Latency Trade-offs....107
8.7.2 Reset Considerations....108
8.8 Related Modules....109
8.9 Testing....109
8.10 References....109
8.11 Navigation....110
9 APB5 Slave (Clock-Gated)....111
9.1 Overview....111
9.1.1 Key Features....111
9.2 Parameters....111
9.2.1 Derived Parameters (do not override)....112
9.3 Ports....113

9.3.1 Clock and Reset....113
9.3.2 Clock Gating Configuration....113
9.3.3 Clock Gating Status....113
9.3.4 APB5 Slave Interface....113
9.3.5 Parity Signals (Optional)....115
9.3.6 Command Interface (to backend)....116
9.3.7 Response Interface (from backend)....116
9.3.8 Status Outputs....117
9.4 Functional Description....117
9.4.1 Block diagram....117
9.4.2 Clock Gating Behavior....118
9.5 Timing Characteristics....118
9.6 Usage Examples....119
9.7 Design Notes....119
9.8 Related Modules....120
9.9 Testing....120
9.10 References....120
9.11 Navigation....121
10 APB5 Slave Stub....122
10.1 Overview....122
10.1.1 Key Features....122
10.2 Parameters....122
10.2.1 Derived Parameters (do not override)....123
10.3 Ports....124

10.3.1 Clock and Reset....124
10.3.2 APB5 Slave Interface....124
10.3.3 Parity Signals (Optional)....125
10.3.4 Command Packet Interface....126
10.3.5 Response Packet Interface....126
10.3.6 Control and Status....126
10.4 Functional Description....127
10.4.1 Block diagram....127
10.4.2 Packet Formats....127
10.4.3 State Descriptions....128
10.4.4 Transaction Flow....129
10.5 Timing Characteristics....129
10.6 Usage Examples....130
10.7 Design Notes....132
10.7.1 Skid Buffers....132
10.7.2 Parity Implementation....132
10.7.3 Wake-up Handling....132
10.8 Related Modules....132
10.9 Testing....132
10.10 References....133
10.11 Navigation....133

1 APB5 (Advanced Peripheral Bus - AMBA 5) Modules

Location: `rtl/amba/apb5/` **Test Location:** `val/amba/` **Status:** Production Ready

1.1 Overview

The APB5 subsystem is a complete implementation of the ARM AMBA 5 APB (Advanced Peripheral Bus) protocol: masters, slaves, monitors, clock domain crossing, and testbench utilities.

APB5 extends APB4 with features aimed at modern SoC designs while staying backward compatible. The simple two-cycle handshake is preserved exactly – what changes is the signal list, which picks up security attributes, wake-up signaling, and improved error handling.

1.1.1 AMBA4 vs AMBA5 Comparison

The table below compares the APB4 and APB5 protocol definitions, and states what this RTL release actually implements. Optional APB5 features that are not implemented are simply absent from the module port lists – there are no tie-off ports to connect.

Feature	APB4	APB5	Implemented here
Basic Protocol	Two-phase handshake	Two-phase handshake	Yes
Protection	PPROT[2:0]	PPROT[2:0] + PNSE	PPROT only – no PNSE port
Wake-up	Not supported	PWAKEUP signal	Yes
User Signals	Not supported	PAUSER, PWUSER, PRUSER, PBUSER	Yes
Atomic Operations	Not supported	PEXCL, PEXOKAY	No – not in this release
Parity	Not supported	Optional signal parity	Yes (`ENABLE_PARITY`)

Feature	APB4	APB5	Implemented here
Error Response	PSLVERR	PSLVERR (enhanced semantics)	Yes

1.1.2 Module Categories

1.1.2.1 Core Protocol Components

Module	Description	Documentation	Status
apb5_master	Full-featured APB5 master with command/response interface	apb5_master.md	Documented
apb5_slave	Complete APB5 slave with buffered cmd/rsp interface	apb5_slave.md	Documented
apb5_slave_cdc	APB5 slave with clock domain crossing support	apb5_slave_cdc.md	Documented
apb5_monitor	Transaction monitoring with 128-bit monitor bus packet + 64-bit side-band timestamp	apb5_monitor.md	Documented

1.1.2.2 Clock-Gated Variants

Module	Description	Documentation	Status
apb5_master_cg	APB5 master with integrated clock gating	apb5_master_cg.md	Documented
apb5_slave_cg	APB5 slave with integrated clock gating	apb5_slave_cg.md	Documented
apb5_slave_cdc_cg	APB5 slave CDC with integrated clock gating	apb5_slave_cdc_cg.md	Documented

1.1.2.3 Testbench Utilities

Module	Description	Documentation	Status
apb5_master_stub	Lightweight APB5 master for testbench integration	apb5_master_stub.md	Documented
apb5_slave_stub	Lightweight APB5 slave for testbench integration	apb5_slave_stub.md	Documented

1.1.2.4 Interconnect

Module	Description	Documentation	Status
apbx_xbar	APB crossbar family; any port may be APB5, and APB4/APB5 may be mixed on one fabric	../apbx/README.md	Documented

The crossbar is documented in its own apbx/ area rather than here, because it is not an APB5-only block — the same RTL covers APB4, APB5, and mixed configurations. An APB5 port on the crossbar instantiates the `apb5_master` / `apb5_slave` boundary IP listed above.

1.1.3 Key Features

1.1.3.1 APB5 Protocol Support

- **PSTRB Support:** Byte-lane strobes for partial writes
- **PPROT Support:** Protection attributes for security-aware systems
- **PWAKEUP Support:** Low-power wake-up signaling
- **User Signals:** PAUSER, PWUSER, PRUSER, PBUSER for sideband data
- **Optional Parity:** Per-byte data parity plus address and control parity (`ENABLE_PARITY`)

Not implemented in this release: PNSE (Non-secure extension) and the PEXCL/PEXOKAY exclusive-access pair. No module in `rtl/amba/apb5/` declares these ports.

1.1.3.2 Clock Domain Crossing

- **Dual-Clock Operation:** APB (pclk) and backend (aclk) domains
- **Safe CDC:** Proper handshake-based clock domain crossing
- **Independent Frequencies:** Backend can run faster or slower than APB

1.1.3.3 Power Management

- **Clock Gating:** Per-module clock gating for power reduction
- **Wake-up Signaling:** PWAKEUP for low-power state exit
- **Idle Detection:** Automatic clock gate when bus is idle

1.1.3.4 Monitoring and Debug

- **Transaction Monitoring:** Real-time protocol monitoring
- **64-bit Monitor Bus:** Standardized packet format (`monbus_packet[63:0]`, no side-band timestamp)
- **Error Detection:** Protocol violations, timeout detection
- **Performance Tracking:** Transaction counting, latency measurement

---

1.2 Functional Description

1.2.1 APB5 Transfer Phases

APB5 uses the same two-phase protocol as APB4:

1. **SETUP Phase:** PSEL asserted, PENABLE low
   - Master presents address, control signals, and write data (if write)
   - Slave prepares for transfer
2. **ACCESS Phase:** PSEL and PENABLE both asserted
   - Slave completes transfer and asserts PREADY when ready
   - For reads, slave presents PRDATA
   - Slave may assert PSLVERR to signal error

1.2.2 APB5 Signal Descriptions

In this implementation the clock and reset ports are named `pclk` and `presetn`, and the bus signals are prefixed `m_apb_` on masters and `s_apb_` on slaves (for example `m_apb_PADDR`, `s_apb_PREADY`).

Signal	Direction	Description
pclk	Input	APB clock
presetn	Input	Active-low reset
PSEL	Master to Slave	Slave select
PENABLE	Master to Slave	Enable (ACCESS phase indicator)

Signal	Direction	Description
PREADY	Slave to Master	Transfer complete
PADDR	Master to Slave	Address bus
PWRITE	Master to Slave	Write enable (1=write, 0=read)
PWDATA	Master to Slave	Write data
PSTRB	Master to Slave	Byte lane strobes
PPROT	Master to Slave	Protection attributes
PWAKEUP	Master to Slave	Wake-up signal (APB5) – see note below
PAUSER	Master to Slave	Address phase user signal (APB5)
PWUSER	Master to Slave	Write data user signal (APB5)
PRDATA	Slave to Master	Read data
PSLVERR	Slave to Master	Error response
PRUSER	Slave to Master	Read data user signal (APB5)
PBUSER	Slave to Master	Write response user signal (APB5)

PWAKEUP direction note: PWAKEUP is a Requester output (IHI 0024E §3.7.2, Table 3-7). `apb5_master` drives `m_apb_PWAKEUP` from a flop, HIGH while a command is pending or a transfer is in flight, raised a cycle ahead of PSEL from an idle bus and held through PREADY. `apb5_slave` takes `s_apb_PWAKEUP` as an input, and the slave `_cg` wrappers count it as clock-gate activity so the gate opens before PSEL arrives. (Before issue #102 item 7 the direction was reversed: slave-driven from a `wakeup_request` port, and captured by the master into `rsp_pwakeup`/`wakeup_pending`. Those three ports are gone.)

PNSE, PEXCL and PEXOKAY are not implemented and do not appear on any module port list.

1.2.3 Optional Parity Signals

Present on all masters and slaves; meaningful only when `ENABLE_PARITY=1`. Names, coverage and widths follow IHI 0024E chapter 5 (Table 5-1): odd parity, one check bit per 8 payload bits, single-bit controls checked by inversion. See apb5_master for the Check Enable terms. `PWAKEUPCHK` travels with `PWAKEUP`, master to slave.

Signal	Width	Direction	Description
PADDRCHK	ceil(ADDR_WIDTH/8)	Master to Slave	Odd parity over PADDR, one bit per 8 bits
PCTRLCHK	1	Master to Slave	Odd parity over PPROT and PWRITE (PNSE not carried)
PSELCHK	1	Master to Slave	Odd parity over PSEL (the spec's PSELxCHK)
PENABLECHK	1	Master to Slave	Odd parity over PENABLE
PWDATACHK	DATA_WIDTH/8	Master to Slave	Odd parity over PWDATA, one bit per byte lane
PSTRBCHK	1	Master to Slave	Odd parity over all of PSTRB
PAUSERCHK	ceil(AUSER_WIDTH/8)	Master to Slave	Odd parity over PAUSER, one bit per 8 bits
PWUSERCHK	ceil(WUSER_WIDTH/8)	Master to Slave	Odd parity over PWUSER, one bit per 8 bits
PRDATACHK	DATA_WIDTH/8	Slave to Master	Odd parity over PRDATA, one bit per byte lane
PREADYCHK	1	Slave to Master	Odd parity over PREADY
PSLVERRCHK	1	Slave to Master	Odd parity over PSLVERR
PWAKEUPCHK	1	Master to Slave	Odd parity over PWAKEUP
PRUSERCHK	ceil(RUSER_WIDTH/8)	Slave to Master	Odd parity over PRUSER, one bit per 8 bits
PBUSERCHK	ceil(BUSER_WIDTH/8)	Slave to Master	Odd parity over PBUSER, one bit per 8 bits

1.3 Usage Examples

1.3.1 Using APB5 Master

```
apb5_master #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .CMD_DEPTH (6),      // command FIFO entries: 2..8 inclusive (any
integer)
    .RSP_DEPTH (6)       // response FIFO entries: 2..8 inclusive (any
integer)
) u_apb5_master (
    .pclk           (clk),
    .presetn        (resetn),

    // Command interface
    .cmd_valid      (cmd_valid),
    .cmd_ready      (cmd_ready),
    .cmd_pwrite     (cmd_pwrite),
    .cmd_paddr      (cmd_paddr),
    .cmd_pwdata     (cmd_pwdata),
    .cmd_pstrb      (cmd_pstrb),
    .cmd_pprot      (cmd_pprot),
    .cmd_pauser     (cmd_pauser),
    .cmd_pwuser     (cmd_pwuser),

    // Response interface
    .rsp_valid      (rsp_valid),
    .rsp_ready      (rsp_ready),
    .rsp_prdata     (rsp_prdata),
    .rsp_pslverr    (rsp_pslverr),
    .rsp_pruser     (rsp_pruser),
    .rsp_pbuser     (rsp_pbuser),

    // APB5 master interface
    .m_apb_PSEL     (psel),
    .m_apb_PENABLE  (penable),
    .m_apb_PREADY   (pready),
    .m_apb_PADDR    (paddr),
    .m_apb_PWRITE   (pwrite),
    .m_apb_PWDATA   (pwdata),
    .m_apb_PSTRB    (pstrb),
    .m_apb_PPROT    (pprot),
    .m_apb_PAUSER   (pauser),
    .m_apb_PWUSER   (pwuser),
    .m_apb_PRDATA   (prdata),
    .m_apb_PSLVERR  (pslverr),
    .m_apb_PWAKEUP  (pwakeup),      // output: Requester wake-up
```

```
    .m_apb_PRUSER   (pruser),
    .m_apb_PBUSER   (pbuser),

    // Parity (tie off / leave open when ENABLE_PARITY=0)
    .m_apb_PADDRCHK      (),
    .m_apb_PCTRLCHK      (),
    .m_apb_PSELCHK       (),
    .m_apb_PENABLECHK    (),
    .m_apb_PWDATACHK     (),
    .m_apb_PSTRBCHK      (),
    .m_apb_PAUSERCHK     (),
    .m_apb_PWUSERCHK     (),
    .m_apb_PRDATACHK     ('0),
    .m_apb_PREADYCHK     (1'b0),
    .m_apb_PSLVERRCHK    (1'b0),
    .m_apb_PWAKEUPCHK    (),
    .m_apb_PRUSERCHK     ('0),
    .m_apb_PBUSERCHK     ('0),
    .parity_error_rdata  (),
    .parity_error_ctrl   ()
);
```

1.3.2 Using APB5 Slave

```
apb5_slave #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .DEPTH(2)
) u_apb5_slave (
    .pclk           (clk),
    .presetn        (resetn),

    // APB5 slave interface
    .s_apb_PSEL     (psel),
    .s_apb_PENABLE  (penable),
    .s_apb_PREADY   (pready),
    .s_apb_PADDR    (paddr),
    .s_apb_PWRITE   (pwrite),
    .s_apb_PWDATA   (pwdata),
    .s_apb_PSTRB    (pstrb),
    .s_apb_PPROT    (pprot),
    .s_apb_PAUSER   (pauser),
    .s_apb_PWUSER   (pwuser),
    .s_apb_PRDATA   (prdata),
    .s_apb_PSLVERR  (pslverr),
    .s_apb_PWAKEUP  (pwakeup),      // input: from the master
    .s_apb_PRUSER   (pruser),
    .s_apb_PBUSER   (pbuser),
```

```
    // Command interface
    .cmd_valid      (cmd_valid),
    .cmd_ready      (cmd_ready),
    .cmd_pwrite     (cmd_pwrite),
    .cmd_paddr      (cmd_paddr),
    .cmd_pwdata     (cmd_pwdata),
    .cmd_pstrb      (cmd_pstrb),
    .cmd_pprot      (cmd_pprot),
    .cmd_pauser     (cmd_pauser),
    .cmd_pwuser     (cmd_pwuser),

    // Response interface
    .rsp_valid      (rsp_valid),
    .rsp_ready      (rsp_ready),
    .rsp_prdata     (rsp_prdata),
    .rsp_pslverr    (rsp_pslverr),
    .rsp_pruser     (rsp_pruser),
    .rsp_pbuser     (rsp_pbuser),


    // Parity (tie off / leave open when ENABLE_PARITY=0)
    .s_apb_PADDRCHK      ('0),
    .s_apb_PCTRLCHK      (1'b0),
    .s_apb_PSELCHK       (1'b0),
    .s_apb_PENABLECHK    (1'b0),
    .s_apb_PWDATACHK     ('0),
    .s_apb_PSTRBCHK      (1'b0),
    .s_apb_PAUSERCHK     ('0),
    .s_apb_PWUSERCHK     ('0),
    .s_apb_PRDATACHK     (),
    .s_apb_PREADYCHK     (),
    .s_apb_PSLVERRCHK    (),
    .s_apb_PWAKEUPCHK    (1'b0),
    .s_apb_PRUSERCHK     (),
    .s_apb_PBUSERCHK     (),
    .parity_error_wdata  (),
    .parity_error_ctrl   ()
);
```

1.4 Design Notes

1.4.1 Command/Response Architecture

All APB5 modules use a command/response interface pattern for backend integration, identical to the APB4 architecture:

Command Interface: - `cmd_valid`, `cmd_ready` - Handshake signals - `cmd_pwrite` - Write/read direction - `cmd_paddr` - Transaction address - `cmd_pwdata` - Write data - `cmd_pstrb` - Byte strobes - `cmd_pprot` - Protection attributes - `cmd_pauser`, `cmd_pwuser` - APB5 user attributes

Response Interface: - `rsp_valid`, `rsp_ready` - Handshake signals - `rsp_prdata` - Read data - `rsp_pslverr` - Error flag - `rsp_pruser`, `rsp_pbuser` - APB5 user attributes

1.4.2 Migration from APB4

APB5 modules are backward compatible with APB4 systems: - Connect APB5 signals to APB4 equivalents - Tie unused APB5 inputs (PWAKEUP, user signals, parity) to default values and leave the corresponding outputs unconnected - On a master, leave `m_apb_PWAKEUP` unconnected if nothing consumes it - On a slave, tie `s_apb_PWAKEUP` to 0 when the Requester has no PWAKEUP - Leave `ENABLE_PARITY=0` (the default), which forces all generated parity outputs and both `parity_error_*` flags to 0

1.5 Related Modules

- **APB4 Modules** - APB4 protocol components
- **AXI5 Modules** - AXI5 protocol components
- **AXIS5 Modules** - AXI5-Stream components
- **GAXI Modules** - Generic AXI utilities

1.6 Testing

All APB5 modules are verified using CocoTB-based testbenches located in `val/amba/`:

```
# Run all APB5 tests
pytest val/amba/test_apb5*.py -v

# Run specific module tests
pytest val/amba/test_apb5_master.py -v
pytest val/amba/test_apb5_slave.py -v
pytest val/amba/test_apb5_slave_cdc.py -v
pytest val/amba/test_apb5_monitor.py -v
```

1.7 References

1.7.1 Specifications

- Arm. *AMBA APB Protocol Specification*, ARM IHI 0024E, Feb. 2023 (APB5 first published in Issue D, Apr. 2021). developer.arm.com/documentation/ihi0024/e – §3.7 “Wake-up signaling”, §3.8 “User signaling”, ch. 5 “Interface parity protection” (Issue D numbers the first two §3.6/§3.7). The `*CHK` signals follow ch. 5: odd parity, Table 5-1 names, coverage sets and one check bit per 8 payload bits (issue #102 item 1). Citation only.
- Arm. *AMBA APB Protocol Specification*, Version 2.0, ARM IHI 0024C, Apr. 2010. developer.arm.com/documentation/ihi0024/c – the APB4 base protocol (ch. 3 “Transfers”, ch. 4 “Operating States”). Citation only.

1.7.2 Papers and textbooks

- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – the valid/ready channel discipline behind the `gaxi_skid_buffer` queues: a buffered stage may stall or add latency without changing what the endpoints compute.
- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 – the Gray-pointer async FIFO (`gaxi_fifo_async`) that carries the command and response across the two clocks.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – the fixed-timeout policy `amba_clock_gate_ctrl` implements: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects”, section “ARM AMBA interconnect protocols suite” (APB bus protocol overview) – textbook treatment (secondary source).

- Chipressian. *Crack the Hardware Interview - from RTL Designers' Perspective*, Part 2 ch. 2 "Handshake Protocols" (valid-ready protocol and valid-ready slices) – textbook treatment of the skid buffer (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`, `rtl/cdc/references/AsyncFifoGrayPointers__Cummings2002.pdf`, `rtl/common/references/DynamicPowerManagementTimeout__Benini2000.pdf`)

1.7.3 Source Code

- RTL: `rtl/amba/apb5/`
- Tests: `val/amba/test_apb5*.py`
- Framework: `bin/TBClasses/components/apb/`

Last Updated: 2026-07-19

1.8 Navigation

- **Back to rtl-amba Index**
- **Back to Main Documentation Index**

2 APB5 Master

Module: `apb5_master.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

2.1 Overview

The APB5 Master module implements a complete AMBA APB5 master interface with all APB5 extensions including user-defined signals, wake-up support, and optional parity for data integrity. It provides command/response buffering for improved system performance.

2.1.1 Key Features

- Full AMBA APB5 protocol compliance
- PAUSER: User-defined request attributes
- PWUSER: User-defined write data attributes
- PRUSER/PBUSER: User-defined response attributes from slave
- PWAKEUP: Requester wake-up output (IHI 0024E §3.7.2), with PWAKEUPCHK
- Optional IHI 0024E ch. 5 interface parity (`*CHK` signals, odd parity)
- Command and response FIFO buffering
- Configurable FIFO depths

2.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address/request user signal width
WUSER_WIDTH	int	4	Write data user signal width
RUSER_WIDTH	int	4	Read data user signal width
BUSER_WIDTH	int	4	Response user signal width

Parameter	Type	Default	Description
CMD_DEPTH	int	6	Command skid-buffer depth in entries; must be 2..8 inclusive (any integer)
RSP_DEPTH	int	6	Response skid-buffer depth in entries; must be 2..8 inclusive (any integer)
ENABLE_PARITY	bit	0	Enable parity generation and checking
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)

`CMD_DEPTH` and `RSP_DEPTH` are passed straight through to `gaxi_skid_buffer`, whose `DEPTH` is a literal entry count – not a log2 exponent. The default of 6 means six buffered entries.

2.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
AW	ADDR_WIDTH
DW	DATA_WIDTH
SW	STRB_WIDTH
PW	PROT_WIDTH
AUW	AUSER_WIDTH
WUW	WUSER_WIDTH
RUW	RUSER_WIDTH
BUW	BUSER_WIDTH
CPW	ADDR_WIDTH + DATA_WIDTH + STRB_WIDTH + PROT_WIDTH + AUSER_WIDTH + WUSER_WIDTH + 1
RPW	DATA_WIDTH + RUSER_WIDTH + BUSER_WIDTH + 2

Two further computed parameters, `CPW` and `RPW`, are exposed for reference and should not be overridden either; with the default widths they evaluate to 80 and 42 respectively.

2.3 Ports

2.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low reset

2.3.2 APB5 Master Interface

Port	Width	Direction	Description
m_apb_PSEL	1	Output	APB select signal
m_apb_PENABLE	1	Output	APB enable signal
m_apb_PADDR	ADDR_WIDTH	Output	APB address
m_apb_PWRITE	1	Output	Write/read indicator (1=write)
m_apb_PWDATA	DATA_WIDTH	Output	Write data
m_apb_PSTRB	STRB_WIDTH	Output	Write byte strobes
m_apb_PPROT	PROT_WIDTH	Output	Protection attributes
m_apb_PAUSER	AUSER_WIDTH	Output	User-defined request attributes
m_apb_PWUSER	WUSER_WIDTH	Output	User-defined write data attributes
m_apb_PRDATA	DATA_WIDTH	Input	Read data from slave
m_apb_PSLVERR	1	Input	Slave error response
m_apb_PR	1	Input	Slave ready

Port	Width	Direction	Description
EADY			
m_apb_PWAKEUP	1	Output	Requester wake-up (IHI 0024E §3.7.2): HIGH while a request is pending or a transfer is in flight
m_apb_PRUSER	RUSER_WIDTH	Input	User-defined read data attributes
m_apb_PBUSER	BUSER_WIDTH	Input	User-defined response attributes

2.3.3 Parity Signals (Optional)

Port	Width	Direction	Description
m_apb_PADDRCHK	ceil(ADDR_WIDTH/8)	Output	Odd parity over PADDR, one bit per 8 bits
m_apb_PCTRLCHK	1	Output	Odd parity over PPROT and PWRITE (PNSE not carried)
m_apb_PSELCHK	1	Output	Odd parity over PSEL (the spec’s PSELxCHK)
m_apb_PENABLECHK	1	Output	Odd parity over PENABLE
m_apb_PWDATACHK	DATA_WIDTH/8	Output	Odd parity over PWDATA, one bit per byte lane
m_apb_PSTRBCHK	1	Output	Odd parity over all of PSTRB
m_apb_PAUSERCHK	ceil(AUSER_WIDTH/8)	Output	Odd parity over PAUSER, one bit per 8 bits
m_apb_PWUSERCHK	ceil(WUSER_WIDTH/8)	Output	Odd parity over PWUSER, one bit per 8 bits
m_apb_PRDATACHK	DATA_WIDTH/8	Input	Odd parity over PRDATA, one bit per byte lane
m_apb_PREADYCHK	1	Input	Odd parity over PREADY

Port	Width	Direction	Description
m_apb_PSLVERRCHK	1	Input	Odd parity over PSLVERR
m_apb_PWAKEUPCHK	1	Output	Odd parity over PWAKEUP
m_apb_PRUSERCHK	ceil(RUSER_WIDTH/8)	Input	Odd parity over PRUSER, one bit per 8 bits
m_apb_PBUSERCHK	ceil(BUSER_WIDTH/8)	Input	Odd parity over PBUSER, one bit per 8 bits

2.3.4 Command Interface

Port	Width	Direction	Description
cmd_valid	1	Input	Command valid
cmd_ready	1	Output	Command ready (FIFO not full)
cmd_pwrite	1	Input	Command write/read
cmd_paddr	ADDR_WIDTH	Input	Command address
cmd_pwdata	DATA_WIDTH	Input	Command write data
cmd_pstrb	STRB_WIDTH	Input	Command write strobes
cmd_pprot	PROT_WIDTH	Input	Command protection attributes
cmd_pauser	AUSER_WIDTH	Input	Command user attributes
cmd_pwuser	WUSER_WIDTH	Input	Command write user attributes

2.3.5 Response Interface

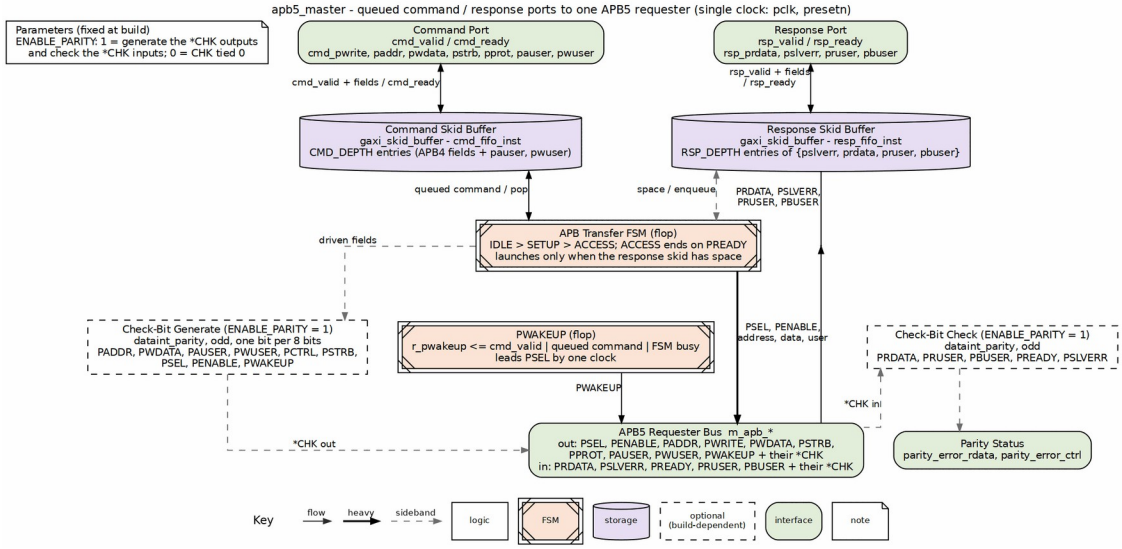
Port	Width	Direction	Description
rsp_valid	1	Output	Response valid
rsp_ready	1	Input	Response ready
rsp_prdata	DATA_WIDTH	Output	Response read data
rsp_pslverr	1	Output	Response error status
rsp_pruser	RUSER_WIDTH	Output	Response read user attributes
rsp_pbuser	BUSER_WIDTH	Output	Response user attributes

2.3.6 Status Outputs

Port	Width	Direction	Description
parity_error_rdata	1	Output	PRDATACHK or PRUSERCHK mismatch on a read completion (tied to 0 when ENABLE_PARITY=0)
parity_error_ctrl	1	Output	PREADYCHK, PSLVERRCHK or PBUSERCHK mismatch (tied to 0 when ENABLE_PARITY=0)

2.4 Functional Description

2.4.1 Block diagram



apb5_master block diagram

Same command / response lanes as `apb4_master`, plus what APB5 adds. The user fields (PAUSER, PWUSER out; PRUSER, PBUSER back) ride through both skids. A PWAKEUP flop rises on `cmd_valid`, a queued command or a busy FSM, so it leads PSEL by one clock. With `ENABLE_PARITY = 1`, `dataint_parity` generates an odd check bit per 8 bits on every driven field and checks the returning *CHK inputs, reporting `parity_error_rdata` and `parity_error_ctrl`; with 0 the CHK outputs are tied low.

```
stateDiagram-v2
    [*] --> IDLE

    IDLE --> SETUP : cmd_fifo_valid
    SETUP --> ACCESS : always
    ACCESS --> IDLE : PREADY & cmd_fifo_empty
    ACCESS --> SETUP : PREADY & !cmd_fifo_empty

    state IDLE {
        note right of IDLE : PSEL=0, PENABLE=0
    }
    state SETUP {
        note right of SETUP : PSEL=1, PENABLE=0
    }
    state ACCESS {
        note right of ACCESS : PSEL=1, PENABLE=1
    }
```

2.4.2 APB5 Extensions

User Signals: - **PAUSER**: Carries user-defined attributes with the address/control phase - **PWUSER**: Carries user-defined attributes with write data - **PRUSER**: Returns user-defined attributes with read data - **PBUSER**: Returns user-defined attributes with the response

Wake-up Support (IHI 0024E §3.7.2): - **PWAKEUP is a Requester output.** The master drives `m_apb_PWAKEUP` from a flop (glitch-free), HIGH while a command is pending (`cmd_valid` at the fub port or a command queued in the command skid) or a transfer is in flight. - A command arriving on an idle bus raises PWAKEUP on the next edge, one cycle before PSEL, as §3.7.2 recommends. - Once PWAKEUP and PSEL are HIGH together, PWAKEUP stays HIGH through the PREADY cycle, as §3.7.2 requires; it drops once nothing further is pending. - A Completer may wait for PWAKEUP before PREADY, so a pending request never leaves it low. The terms are this side's pending work only, never the peer's PREADY.

Parity Protection: - Optional IHI 0024E chapter 5 interface parity: `*CHK` signals, odd parity, one check bit per 8 payload bits - Detects a single-bit error in any covered byte or control signal

See Parity Implementation for the exact coverage of each check signal.

2.5 Timing Characteristics

2.5.1 Basic Write Transaction

Timing diagram pending. The signals and sequence this scenario exercises:

- PCLK
- PSEL
- PENABLE
- PADDR
- PWRITE (high)
- PWDATA
- PSTRB
- PAUSER
- PWUSER
- PREADY
- PSLVERR

2.5.2 Basic Read Transaction

Timing diagram pending. The signals and sequence this scenario exercises:

- PCLK
- PSEL
- PENABLE
- PADDR
- PWRITE (low)
- PAUSER
- PREADY
- PRDATA
- PRUSER
- PSLVERR

2.5.3 Wake-up Signal Handling

Timing diagram pending. The signals and sequence this scenario exercises:

- PCLK
- cmd_valid on an idle bus
- PWAKEUP rising one cycle before PSEL
- PWAKEUP held through the PREADY cycle, then dropped when idle

---

2.6 Usage Examples

```
apb5_master #(
    .ADDR_WIDTH     (32),
    .DATA_WIDTH     (32),
    .AUSER_WIDTH    (4),
    .WUSER_WIDTH    (4),
    .RUSER_WIDTH    (4),
    .BUSER_WIDTH    (4),
    .CMD_DEPTH      (4),
    .RSP_DEPTH      (4),
    .ENABLE_PARITY  (0)
) u_apb5_master (
    .pclk           (apb_clk),
    .presetn        (apb_rst_n),
```

```
// APB5 master interface
.m_apb_PSEL     (m_apb_psel),
.m_apb_PENABLE  (m_apb_penable),
.m_apb_PADDR    (m_apb_paddr),
.m_apb_PWRITE   (m_apb_pwrite),
.m_apb_PWDATA   (m_apb_pwdata),
.m_apb_PSTRB    (m_apb_pstrb),
.m_apb_PPROT    (m_apb_pprot),
.m_apb_PAUSER   (m_apb_pauser),
.m_apb_PWUSER   (m_apb_pwuser),
.m_apb_PRDATA   (m_apb_prdata),
.m_apb_PSLVERR  (m_apb_pslverr),
.m_apb_PREADY   (m_apb_pready),
.m_apb_PWAKEUP  (m_apb_pwakeup),   // output: Requester wake-up
.m_apb_PRUSER   (m_apb_pruser),
.m_apb_PBUSER   (m_apb_pbuser),

// Command interface
.cmd_valid      (cmd_valid),
.cmd_ready      (cmd_ready),
.cmd_pwrite     (cmd_write),
.cmd_paddr      (cmd_addr),
.cmd_pwdata     (cmd_wdata),
.cmd_pstrb      (cmd_strb),
.cmd_pprot      (cmd_prot),
.cmd_pauser     (cmd_auser),
.cmd_pwuser     (cmd_wuser),

// Response interface
.rsp_valid      (rsp_valid),
.rsp_ready      (rsp_ready),
.rsp_prdata     (rsp_rdata),
.rsp_pslverr    (rsp_error),
.rsp_pruser     (rsp_ruser),
.rsp_pbuser     (rsp_buser),

// Parity interface (unused here because ENABLE_PARITY=0)
.m_apb_PADDRCHK      (),
.m_apb_PCTRLCHK      (),
.m_apb_PSELCHK       (),
.m_apb_PENABLECHK    (),
.m_apb_PWDATACHK     (),
.m_apb_PSTRBCHK      (),
.m_apb_PAUSERCHK     (),
.m_apb_PWUSERCHK     (),
.m_apb_PRDATACHK     ('0),
.m_apb_PREADYCHK     (1'b0),
.m_apb_PSLVERRCHK    (1'b0),
```

```
    .m_apb_PWAKEUPCHK    (),
    .m_apb_PRUSERCHK     ('0),
    .m_apb_PBUSERCHK     ('0),
    .parity_error_rdata  (),
    .parity_error_ctrl   ()
);
```

2.7 Design Notes

2.7.1 APB5 vs APB4 Differences

Feature	APB4	APB5	Implemented here
User signals	None	PAUSER, PWUSER, PRUSER, PBUSER	Yes
Wake-up	None	PWAKEUP	Yes (master output, §3.7.2)
Parity	None	Optional signal parity	Yes (ENABLE_PARITY)
Non-secure extension	None	PNSE	No – no port
Exclusive access	None	PEXCL, PEXOKAY	No – no ports

2.7.2 FIFO Sizing

- Command depth should match the expected command burst length
- Response depth should match to prevent backpressure
- CMD_DEPTH / RSP_DEPTH are entry counts, restricted to 2..8 inclusive by the underlying gaxi_skid_buffer

2.7.3 Parity Implementation

When ENABLE_PARITY=1 the master drives the requester check signals (PADDRCHK to PWUSERCHK below) and checks the completer ones. With ENABLE_PARITY=0 the outputs are tied LOW and the inputs are ignored.

Parity follows AMBA APB IHI 0024E chapter 5 (§5.3, Table 5-1):

- **Odd parity.** A payload slice plus its check bit always hold an odd number of ones. On a single-bit critical control the check bit is therefore the inversion of the signal.
- **One check bit per 8 payload bits.** Bit `[n]` of a multi-bit check signal covers payload bits `[8n+7:8n]`; when the payload is not a whole number of bytes, the top check bit covers the bits that remain. `PADDRCHK` on a 12-bit address is therefore 2 bits: `[7:0]` and `[11:8]`.
- **Coverage sets.** `PCTRLCHK` covers `PPROT` and `PWRITE` (and `PNSE`, which this module does not carry, so it counts as LOW). `PSTRB` has its own single `PSTRBCHK`.
- Each check bit is computed by an `rtl/common/dataint_parity` instance with `CHUNKS = 1` and `parity_type = 0` (odd), one instance per spec byte, so the spec's short final byte comes out at any width.

Check signal	Covers	Width	Check Enable (Table 5-1)
PADDRCHK	PADDR	ceil(ADDR_WIDTH/8)	PSEL
PCTRLCHK	PPROT, PWRITE	1	PSEL
PSELCHK	PSEL (spec PSELxCHK)	1	PRESETn
PENABLECHK	PENABLE	1	PSEL
PWDATACHK	PWDATA	DATA_WIDTH/8	PSEL & PWRITE
PSTRBCHK	PSTRB	1	PSEL & PWRITE
PAUSERCHK	PAUSER	ceil(AUSER_WIDTH/8)	PSEL
PWUSERCHK	PWUSER	ceil(WUSER_WIDTH/8)	PSEL & PWRITE

Check signal	Covers	Width	Check Enable (Table 5-1)
PRDATACHK	PRDATA	DATA_WIDTH/8	PSEL & PENABLE & PREADY & !PWRITE
PREADYCHK	PREADY	1	PSEL & PENABLE
PSLVERRCHK	PSLVERR	1	PSEL & PENABLE & PREADY
PWAKEUPCHK	PWAKEUP	1	PRESETn
PRUSERCHK	PRUSER	ceil(RUSER_WIDTH/8)	PSEL & PENABLE & PREADY & !PWRITE
PBUSERCHK	PBUSER	ceil(BUSER_WIDTH/8)	PSEL & PENABLE & PREADY

`PWAKEUPCHK` travels with `PWAKEUP`: both are Requester outputs (IHI 0024E §3.7.2, Table 3-7), generated by the master and checked by the slave.

The error flags are combinational and assert only while the failing check's Check Enable term holds:

- `parity_error_rdata`: `PRDATACHK` or `PRUSERCHK` wrong on a read completion.
- `parity_error_ctrl`: `PREADYCHK`, `PSLVERRCHK` or `PBUSERCHK` wrong.
- `PWAKEUPCHK` is generated here, so the master does not check it.

The master reports a mismatch but does not abort or retry the transfer; the spec leaves that policy to the system (§5.4). Parity adds combinational logic but no pipeline stages, so it costs no latency.

2.8 Related Modules

- **APB5 Slave** - APB5 slave interface
- **APB5 Master CG** - Clock-gated variant
- **APB5 Monitor** - Protocol monitor
- **APB4 Master** - APB4 version for comparison

2.9 Testing

`val/amba/test_apb5_master.py` exercises this module. It collects 5 parameter cases at the default `REG_LEVEL`; the two `ENABLE_PARITY=1` cases also run `cocotb_test_apb5_master_parity`, a directed IHI 0024E ch. 5 check (generated `*CHK` values, flag-per-check error injection, Check Enable gating). The `formal/amba/apb5_master` proof checks the same parity rules with `ENABLE_PARITY=1`.

```
source env_python
pytest val/amba/test_apb5_master.py -v
```

2.10 References

- Arm. *AMBA APB Protocol Specification*, ARM IHI 0024E, Feb. 2023 (APB5 first published in Issue D, Apr. 2021). developer.arm.com/documentation/ihi0024/e – §3.7 “Wake-up signaling” (`PWAKEUP`), §3.8 “User signaling” (`PAUSER`/`PWUSER`/`PRUSER`/`PBUSER`), ch. 5 “Interface parity protection” (`ENABLE_PARITY`); transfers and operating states (ch. 3-4) are as in APB4. The `*CHK` signals follow §5.3 and Table 5-1: odd parity, one check bit per 8 payload bits, `PCTRLCHK` over `PPROT`/`PWRITE` and a separate `PSTRBCHK` (issue #102 item 1). Citation only.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – the valid/ready channel discipline behind the `gaxi_skid_buffer` queues: a buffered stage may stall or add latency without changing what the endpoints compute.
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects”, section “ARM AMBA interconnect protocols suite” (APB bus protocol overview) – textbook treatment (secondary source).
- Chipressian. *Crack the Hardware Interview - from RTL Designers’ Perspective*, Part 2 ch. 2 “Handshake Protocols” (valid-ready protocol and valid-ready slices) – textbook treatment of the skid buffer (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`)

2.11 Navigation

- ← Back to APB5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

3 APB5 Master (Clock-Gated)

Module: `apb5_master_cg.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

3.1 Overview

Clock-gated variant of the APB5 Master module. Wraps the base `apb5_master` with clock gating control logic to reduce dynamic power consumption during idle periods.

3.1.1 Key Features

- All APB5 Master features (see apb5_master)
- Automatic clock gating during idle periods
- Configurable idle threshold before gating
- Registered wake-up on new activity, through a glitch-free ICG cell
- Power consumption reduction for low-duty-cycle applications

3.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address/request user signal width
WUSER_WIDTH	int	4	Write data user signal width
RUSER_WIDTH	int	4	Read data user signal width
BUSER_WIDTH	int	4	Response user signal width
CMD_DEPTH	int	6	Command FIFO depth
RSP_DEPTH	int	6	Response FIFO depth
ENABLE_PARITY	bit	0	Enable APB5 parity generation and checking

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter (max idle = 2^N-1 cycles)

All other parameters are inherited from apb5_master.

3.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
STRB_WIDTH	DATA_WIDTH / 8
AW	ADDR_WIDTH
DW	DATA_WIDTH
SW	STRB_WIDTH
PW	PROT_WIDTH
AUW	AUSER_WIDTH
WUW	WUSER_WIDTH
RUW	RUSER_WIDTH
BUW	BUSER_WIDTH
ICW	CG_IDLE_COUNT_WIDTH
CPW	AW + DW + SW + PW + AUW + WUW + 1
RPW	DW + RUW + BUW + 2

3.3 Ports

3.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low reset

3.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating (0=disabled, clock always runs)
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating

3.3.3 Clock Gating Status

Port	Width	Direction	Description
cg_gating	1	Output	High while the internal clock is gated off
cg_idle	1	Output	Activity terms quiet (registered ~wakeup).

There is no cumulative gated-cycle counter port on this module. If a gated-cycle total is needed, count `cg_gating` in the integrating logic on the ungated `pclk`.

All ports of apb5_master – including the parity signals, `m_apb_PWAKEUP`, `parity_error_rdata` and `parity_error_ctrl` – are present unchanged and pass straight through to the wrapped core.

3.3.4 APB5 Master Interface

Port	Width	Direction	Description
m_apb_PSEL	1	Output	APB select signal
m_apb_PENABLE	1	Output	APB enable signal
m_apb_PADDR	AW	Output	APB address
m_apb_PWRITE	1	Output	Write/read indicator (1=write)
m_apb_PWDATA	DW	Output	Write data

Port	Width	Direction	Description
m_apb_PSTRB	SW	Output	Write byte strobes
m_apb_PPROT	PW	Output	Protection attributes
m_apb_PAUSER	AUW	Output	User-defined request attributes
m_apb_PWUSER	WUW	Output	User-defined write data attributes
m_apb_PRDATA	DW	Input	Read data from slave
m_apb_PSLVERR	1	Input	Slave error response
m_apb_PREADY	1	Input	Slave ready
m_apb_PWAKEUP	1	Output	Requester wake-up (IHI 0024E §3.7.2), driven by the wrapped master
m_apb_PRUSER	RUW	Input	User-defined read data attributes
m_apb_PBUSER	BUW	Input	User-defined response attributes

3.3.5 Parity Signals (Optional)

Port	Width	Direction	Description
m_apb_PADDRCHK	ceil(ADDR_WIDTH/8)	Output	Odd parity over PADDR, one bit per 8 bits
m_apb_PCTRLCHK	1	Output	Odd parity over PPROT and PWRITE (PNSE not carried)
m_apb_PSELCHK	1	Output	Odd parity over PSEL (the spec’s PSELxCHK)
m_apb_PENABLECHK	1	Output	Odd parity over PENABLE

Port	Width	Direction	Description
m_apb_PWDATACHK	DATA_WIDTH/8	Output	Odd parity over PWDATA, one bit per byte lane
m_apb_PSTRBCHK	1	Output	Odd parity over all of PSTRB
m_apb_PAUSERCHK	ceil(AUSER_WIDTH/8)	Output	Odd parity over PAUSER, one bit per 8 bits
m_apb_PWUSERCHK	ceil(WUSER_WIDTH/8)	Output	Odd parity over PWUSER, one bit per 8 bits
m_apb_PRDATACHK	DATA_WIDTH/8	Input	Odd parity over PRDATA, one bit per byte lane
m_apb_PREADYCHK	1	Input	Odd parity over PREADY
m_apb_PSLVERRCHK	1	Input	Odd parity over PSLVERR
m_apb_PWAKEUPCHK	1	Output	Odd parity over PWAKEUP
m_apb_PRUSERCHK	ceil(RUSER_WIDTH/8)	Input	Odd parity over PRUSER, one bit per 8 bits
m_apb_PBUSERCHK	ceil(BUSER_WIDTH/8)	Input	Odd parity over PBUSER, one bit per 8 bits

3.3.6 Command Interface

Port	Width	Direction	Description
cmd_valid	1	Input	Command valid
cmd_ready	1	Output	Command ready (FIFO not full)
cmd_pwrite	1	Input	Command write/read
cmd_paddr	AW	Input	Command address
cmd_pwdata	DW	Input	Command

Port	Width	Direction	Description
			write data
cmd_pstrb	SW	Input	Command write strobes
cmd_pprot	PW	Input	Command protection attributes
cmd_pauser	AUW	Input	Command user attributes
cmd_pwuser	WUW	Input	Command write user attributes

3.3.7 Response Interface

Port	Width	Direction	Description
rsp_valid	1	Output	Response valid
rsp_ready	1	Input	Response ready
rsp_prdata	DW	Output	Response read data
rsp_pslverr	1	Output	Response error status
rsp_pruser	RUW	Output	Response read user attributes
rsp_pbuser	BUW	Output	Response user attributes

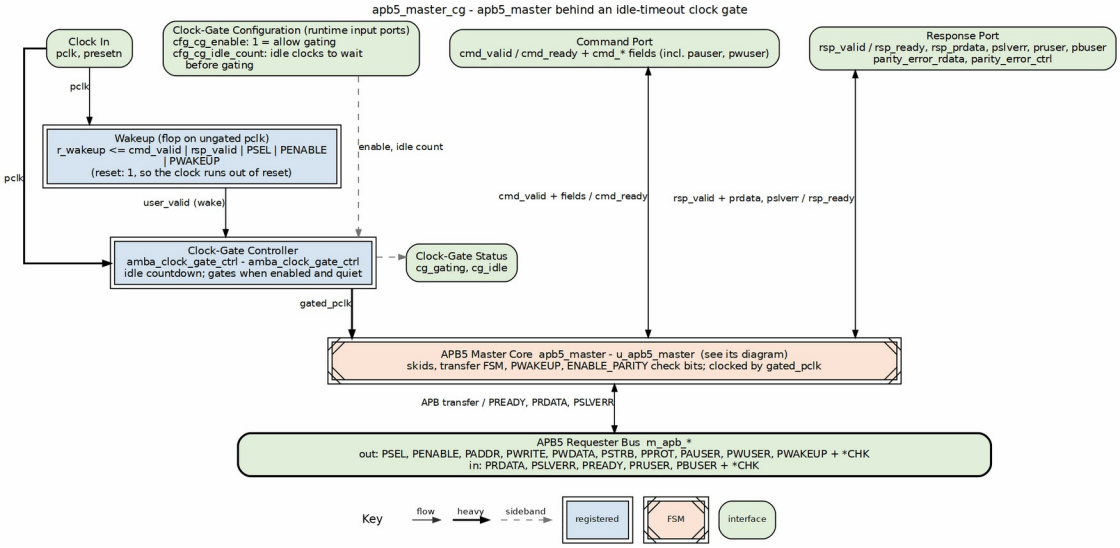
3.3.8 Status Outputs

Port	Width	Direction	Description
parity_error_rdata	1	Output	PRDATACHK or PRUSERCHK mismatch on a read completion (tied to 0 when ENABLE_PARITY=0)
parity_erro	1	Output	PREADYCHK,

Port	Width	Direction	Description
r_ctrl			PSLVERRCHK or PBUSERCHK mismatch (tied to 0 when ENABLE_PARITY=0)

3.4 Functional Description

3.4.1 Block diagram



apb5_master_cg block diagram

The wrapped `apb5_master` is one box (see its diagram). The variant adds the clock path (heavy): a wakeup flop on the ungated `pclk` watches `cmd_valid`, `rsp_valid`, PSEL, PENABLE and PWAKEUP, and `amba_clock_gate_ctrl` stops the core's clock after `cfg_cg_idle_count` quiet clocks when `cfg_cg_enable` is set.

3.4.2 Clock Gating Behavior

```
stateDiagram-v2
    [*] --> RUNNING

    RUNNING --> COUNTING : No activity
    COUNTING --> RUNNING : Activity detected
    COUNTING --> GATED : count >= threshold
    GATED --> RUNNING : cmd_valid or activity

    state RUNNING {
```

```
        note right of RUNNING : Clock running
    }
    state COUNTING {
        note right of COUNTING : Counting idle cycles
    }
    state GATED {
        note right of GATED : Clock stopped
    }
```

3.4.2.1 Activity Detection and Wake-up Latency

The wrapper keeps the clock running whenever any of the following is high:

```
cmd_valid || rsp_valid || m_apb_PSEL || m_apb_PENABLE || m_apb_PWAKEUP
```

`m_apb_PWAKEUP` is this master's own Requester wake-up output (IHI 0024E §3.7.2). It covers a command still queued in the command skid after the fub handshake, which no other term sees. None of the terms is a peer READY.

That term is registered into `r_wakeup` on the ungated `pclk`, and `amba_clock_gate_ctrl` registers it once more before it reaches the gating condition. Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. APB5 is a **two-stage** family, so the first gated-clock rising edge available to the wrapped `apb5_master` arrives **3 ungated `pclk` cycles** after activity asserts. Nothing is lost during those cycles – the core is simply held static, and the APB transfer stretches by that many wait states.

The actual gating element is an ICG (integrated clock gating) cell instantiated by `clock_gate_ctrl`, not a bare AND gate, so enable changes are glitch-free. Note that ICG cells are an ASIC-library primitive; on FPGA targets a clock-enable approach should be used instead.

Gating engages `cfg_cg_idle_count + 1` ungated `pclk` cycles after the internal wakeup deasserts, which is `cfg_cg_idle_count + 3` cycles after the last bus activity, because APB5 adds two register stages ahead of the ICG enable. With the default `CG_IDLE_COUNT_WIDTH=4` the maximum programmable idle threshold is 15 cycles.

3.5 Timing Characteristics

Timing diagram pending. The signals and sequence this scenario exercises:

- pclk
- gated_pclk
- cmd_valid
- cfg_cg_idle_count

- idle_counter
- cg_gating
- Transaction before/after gating

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

3.6 Usage Examples

```
apb5_master_cg #(
    .ADDR_WIDTH         (32),
    .DATA_WIDTH         (32),
    .AUSER_WIDTH        (4),
    .WUSER_WIDTH        (4),
    .CG_IDLE_COUNT_WIDTH(4)
) u_apb5_master_cg (
    .pclk               (apb_clk),
    .presetn            (apb_rst_n),

    // Clock gating configuration
    .cfg_cg_enable      (1'b1),
    .cfg_cg_idle_count  (4'd8),    // Gate after 8 idle cycles

    // Clock gating status
    .cg_gating   (master_clk_gated),

    // APB5 and command/response interfaces
    // ... (same as apb5_master)
);
```

3.7 Design Notes

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into `user_valid` pins this block

permanently awake and defeats gating entirely – silently, because function is unaffected. Ten wrappers in this repository shipped that way and nothing failed until someone measured. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating`. Canonical rule: `vault/handbook/design/clock-gating-activity-terms.md`.

`cfg_cg_enable` is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only; the datapath and any monitor enables are forwarded untouched. With it low the clock free-runs and this module behaves exactly like its base.

Gating latency. The clock stops `cfg_cg_idle_count` + 2 cycles after the last bus activity – the idle counter, plus one for the `r_wakeup` flop. Size the idle count against your traffic's inter-burst gap: too small and the block wakes constantly, too large and it never gates.

Cost. Five flops: `r_wakeup` plus `r_idle_counter` at `IDLE_CNTR_WIDTH`, scaling as 1 + `CG_IDLE_COUNT_WIDTH`. The ICG itself is a latch or BUFGCE, not fabric flops.

3.7.1 Power Savings

The figures below are first-order expectations derived from the fraction of cycles the clock is gated off; they are analytical estimates, not measured silicon or post-layout power numbers. Actual savings depend on the technology library, the ICG cell, and the clock-tree share of total dynamic power.

Traffic Pattern	Duty Cycle	Estimated Dynamic Savings
Burst	30%	35-40%
Mixed	50%	20-25%
Continuous	90%+	<5%

3.8 Related Modules

- **APB5 Master** - Base module documentation
- **APB5 Slave CG** - Clock-gated slave
- **Clock Gating Guide** - General clock gating concepts

3.9 Testing

`val/amba/test_apb5_master_cg.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_apb5_master_cg.py -v
```

3.10 References

- Arm. *AMBA APB Protocol Specification*, ARM IHI 0024E, Feb. 2023 (APB5 first published in Issue D, Apr. 2021). developer.arm.com/documentation/ihi0024/e – ch. 3 “Transfers”, ch. 4 “Operating States”, §3.7 “Wake-up signaling” and §3.8 “User signaling”: the protocol of the wrapped APB5 block. Citation only.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – the fixed-timeout policy `amba_clock_gate_ctrl` implements: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. CAD*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based gated clock (`icg`) at the bottom of the gating chain.
- Roy, S. *Advanced Digital System Design*, Springer, ch. 15 “Low-Power Digital System Design”, §15.5.5 “Logic Level Power Down” – textbook treatment of clock gating (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/common/references/DynamicPowerManagementTimeout__Benini2000.pdf`, `rtl/common/references/LatchBasedClockGating__Benini1996.pdf`)

3.11 Navigation

- **← Back to APB5 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

4 APB5 Master Stub

Module: `apb5_master_stub.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

4.1 Overview

The APB5 Master Stub provides a simplified packed-data interface for driving APB5 transactions. It wraps the full `apb5_master` module with packet-based command/response interfaces, making it ideal for testbenches and integration scenarios where a simpler interface is preferred.

4.1.1 Key Features

- Packed command/response packet interface
- Wraps fully-tested `apb5_master` internally
- All APB5 extensions (PAUSER, PWUSER, PRUSER, PBUSER)
- PWAKEUP Requester wake-up output (IHI 0024E §3.7.2)
- Optional IHI 0024E ch. 5 interface parity (`*CHK` signals, odd parity)
- First/last markers for transaction tracking

4.2 Parameters

Parameter	Type	Default	Description
CMD_DEPTH	int	6	Command skid-buffer depth in entries; 2..8 inclusive (any integer)
RSP_DEPTH	int	6	Response skid-buffer depth in entries; 2..8 inclusive (any integer)
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address user signal width
WUSER_WIDTH	int	4	Write user signal width
RUSER_WIDTH	int	4	Read user signal width

Parameter	Type	Default	Description
BUSER_WIDTH	int	4	Response user signal width
ENABLE_PARITY	bit	0	Enable parity signals
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width
CMD_PACKET_WIDTH	int	(calculated)	Total command packet width
RESP_PACKET_WIDTH	int	(calculated)	Total response packet width

`CMD_PACKET_WIDTH` and `RESP_PACKET_WIDTH` are computed from the width parameters and must not be overridden:

```
CMD_PACKET_WIDTH  = ADDR_WIDTH + DATA_WIDTH + STRB_WIDTH + PROT_WIDTH
                  + AUSER_WIDTH + WUSER_WIDTH + 1 + 1 + 1
                    // the three single bits are pwrite, first, last

RESP_PACKET_WIDTH = DATA_WIDTH + RUSER_WIDTH + BUSER_WIDTH + 1 + 1 + 1
                    // the three single bits are pslverr, first, last
```

With all defaults (32-bit address and data, 4-bit strobe, 3-bit protection, 4-bit user fields) this gives `CMD_PACKET_WIDTH = 82` and `RESP_PACKET_WIDTH = 43`. (Before issue #102 item 7 the packet also carried a `pwakeup` bit captured from a slave-driven PWAKEUP; PWAKEUP is now a master output, so that bit is gone and the packet is one bit narrower.)

4.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
PW	PROT_WIDTH
CPW	CMD_PACKET_WIDTH
RPW	RESP_PACKET_WIDTH

4.3 Ports

4.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB reset (active low)

4.3.2 APB5 Master Interface

Port	Width	Direction	Description
m_apb_PSEL	1	Output	APB select signal
m_apb_PENABLE	1	Output	APB enable signal
m_apb_PADDR	ADDR_WIDTH	Output	APB address
m_apb_PWRITE	1	Output	Write/read indicator
m_apb_PWDATA	DATA_WIDTH	Output	Write data
m_apb_PSTRB	STRB_WIDTH	Output	Write byte strobes
m_apb_PPROT	PROT_WIDTH	Output	Protection attributes
m_apb_PAUSER	AUSER_WIDTH	Output	User request attributes
m_apb_PWUSER	WUSER_WIDTH	Output	User write attributes
m_apb_PRDATA	DATA_WIDTH	Input	Read data from slave
m_apb_PSLVERR	1	Input	Slave error response
m_apb_PREADY	1	Input	Slave ready
m_apb_PWAKE	1	Output	Requester

Port	Width	Direction	Description
UP			wake-up (IHI 0024E §3.7.2)
m_apb_PRUSER	RUSER_WIDTH	Input	User read attributes
m_apb_PBUSER	BUSER_WIDTH	Input	User response attributes

4.3.3 Parity Signals (Optional)

Present unconditionally; meaningful only when `ENABLE_PARITY=1`. Semantics are identical to apb5_master.

Port	Width	Direction	Description
m_apb_PADDRCHK	ceil(ADDR_WIDTH/8)	Output	Odd parity over PADDR, one bit per 8 bits
m_apb_PCTRLCHK	1	Output	Odd parity over PPROT and PWRITE (PNSE not carried)
m_apb_PSELCHK	1	Output	Odd parity over PSEL (the spec's PSELxCHK)
m_apb_PENABLECHK	1	Output	Odd parity over PENABLE
m_apb_PWDATACHK	DATA_WIDTH/8	Output	Odd parity over PWDATA, one bit per byte lane
m_apb_PSTRBCHK	1	Output	Odd parity over all of PSTRB
m_apb_PAUSERCHK	ceil(AUSER_WIDTH/8)	Output	Odd parity over PAUSER, one bit per 8 bits
m_apb_PWUSERCHK	ceil(WUSER_WIDTH/8)	Output	Odd parity over PWUSER, one bit per 8 bits
m_apb_PRDATACHK	DATA_WIDTH/8	Input	Odd parity over PRDATA, one bit per byte lane
m_apb_PREADYCHK	1	Input	Odd parity over PREADY

Port	Width	Direction	Description
m_apb_PSLVERRCHK	1	Input	Odd parity over PSLVERR
m_apb_PWAKEUPCHK	1	Output	Odd parity over PWAKEUP
m_apb_PRUSERCHK	ceil(RUSER_WIDTH/8)	Input	Odd parity over PRUSER, one bit per 8 bits
m_apb_PBUSERCHK	ceil(BUSER_WIDTH/8)	Input	Odd parity over PBUSER, one bit per 8 bits

4.3.4 Command Packet Interface

Port	Width	Direction	Description
cmd_valid	1	Input	Command packet valid
cmd_ready	1	Output	Ready to accept command
cmd_data	CMD_PACKET_WIDTH	Input	Packed command data

4.3.5 Response Packet Interface

Port	Width	Direction	Description
rsp_valid	1	Output	Response packet valid
rsp_ready	1	Input	Ready to accept response
rsp_data	RESP_PACKET_WIDTH	Output	Packed response data

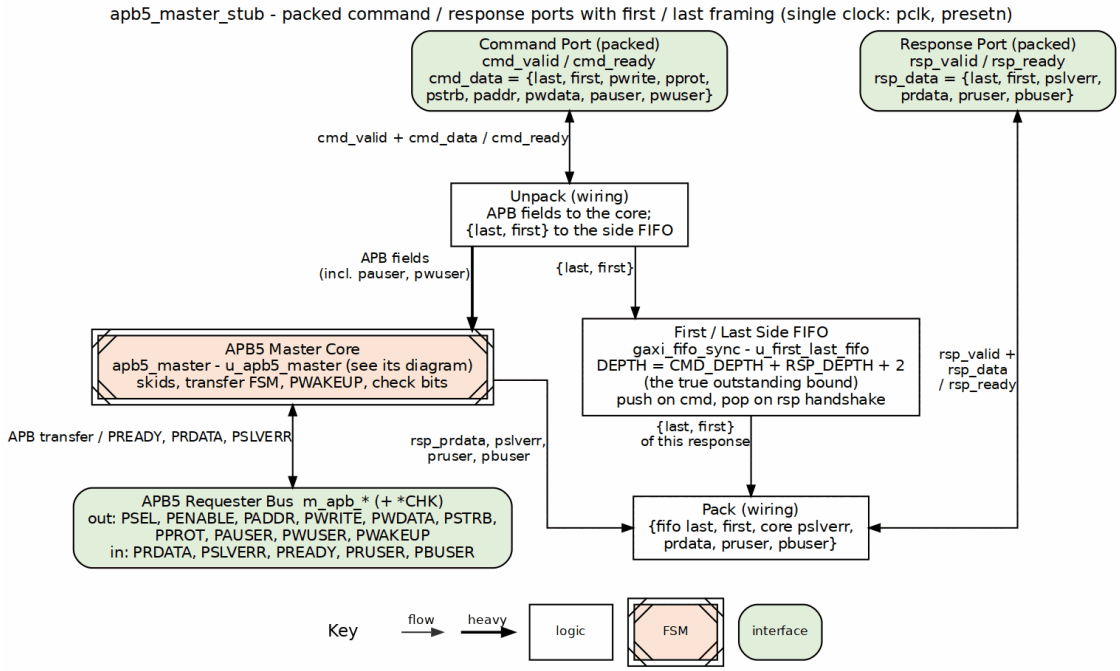
4.3.6 Status Outputs

Port	Width	Direction	Description
parity_error_rdata	1	Output	PRDATACHK or PRUSERCHK mismatch on

Port	Width	Direction	Description
			a read completion
parity_error_ctrl	1	Output	PREADYCHK, PSLVERRCHK or PBUSERCHK mismatch

4.4 Functional Description

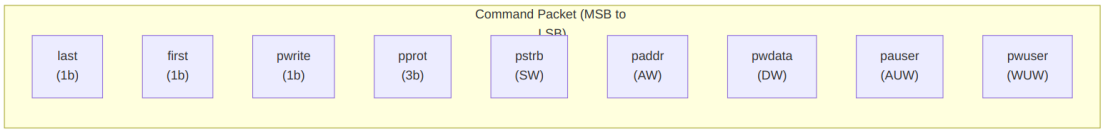
4.4.1 Block diagram



apb5_master_stub block diagram

A packed-port wrapper around `apb5_master` (one box; see its diagram). Unpack sends the APB fields to the core; Pack builds `rsp_data` from the core's response and `{last, first}`. `{last, first}` do not come from the live `cmd_data`: a framing side FIFO (`gaxi_fifo_sync`, `u_first_last_fifo`) pushes them on every command handshake and pops them on every response handshake, so each response carries the framing of the command that produced it even when later commands are already accepted. Its depth, `CMD_DEPTH + RSP_DEPTH + 2`, is the true outstanding bound (command skid, the transfer in flight, response skid), so it never overflows. This is the `apb4_master_stub` fix (TASK-067), ported for issue #111; `test_apb5_master_stub_framing` pins it.

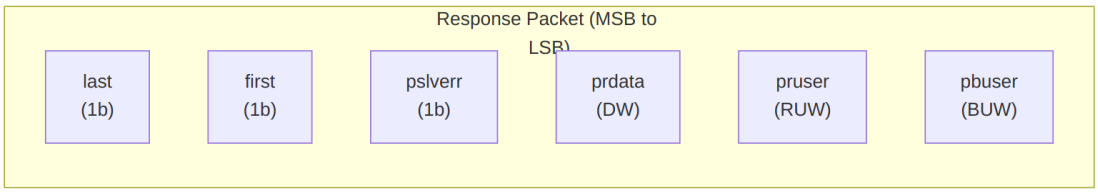
4.4.2 Packet Formats



Command Packet Structure

Bit Positions:

```
cmd_data = {last, first, pwrite, pprot, pstrb, paddr, pwdata, pauser,
pwuser}
```

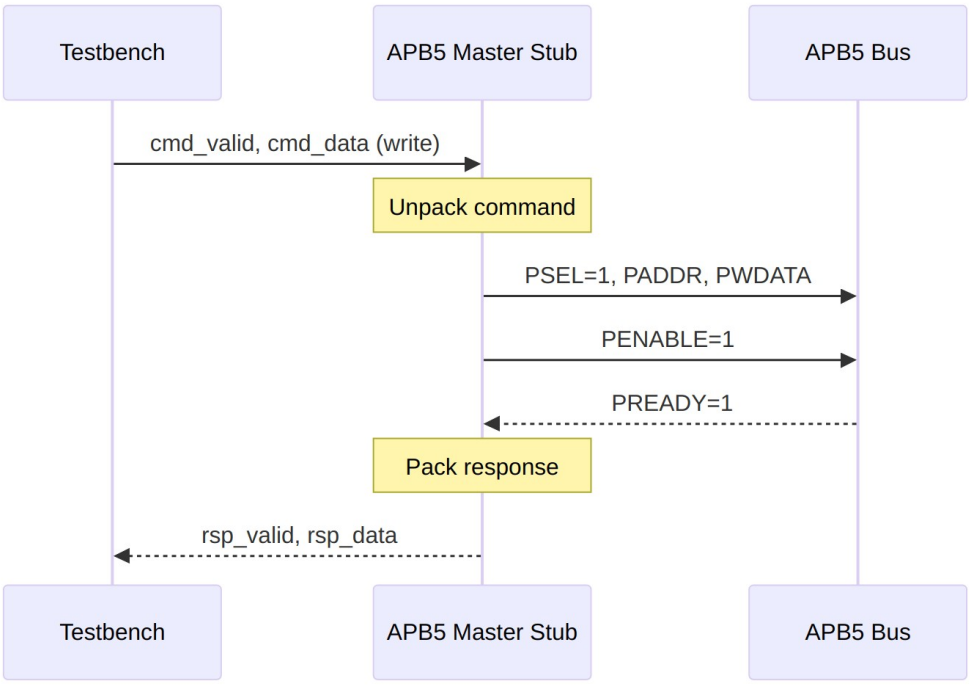


Response Packet Structure

Bit Positions:

```
rsp_data = {last, first, pslverr, prdata, pruser, pbuser}
```

4.4.3 Transaction Flow



Write Transaction

4.5 Timing Characteristics

Timing diagram pending. The signals and sequence this scenario exercises:

- pclk
- cmd_valid, cmd_ready, cmd_data
- APB signals (PSEL, PENABLE, PADDR, PWDATA, PREADY)
- rsp_valid, rsp_ready, rsp_data
- Packet-to-APB timing relationship

This module is **purely combinational** – it contains no `always_ff` and no latch, so it holds no state and adds no clock cycles. Its outputs settle a propagation delay after its inputs, and it introduces no latency into a pipeline that instantiates it.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

4.6 Usage Examples

```
apb5_master_stub #(
    .CMD_DEPTH      (6),
    .RSP_DEPTH      (6),
    .ADDR_WIDTH     (32),
    .DATA_WIDTH     (32),
    .AUSER_WIDTH    (4),
    .WUSER_WIDTH    (4),
    .RUSER_WIDTH    (4),
    .BUSER_WIDTH    (4),
    .ENABLE_PARITY  (0)
) u_apb5_master_stub (
    .pclk           (apb_clk),
    .presetn        (apb_rst_n),

    // APB5 master interface
    .m_apb_PSEL     (m_apb_psel),
    .m_apb_PENABLE  (m_apb_penable),
    .m_apb_PADDR    (m_apb_paddr),
    .m_apb_PWRITE   (m_apb_pwrite),
    .m_apb_PWDATA   (m_apb_pwdata),
    .m_apb_PSTRB    (m_apb_pstrb),
    .m_apb_PPROT    (m_apb_pprot),
    .m_apb_PAUSER   (m_apb_pauser),
    .m_apb_PWUSER   (m_apb_pwuser),
    .m_apb_PRDATA   (m_apb_prdata),
    .m_apb_PSLVERR  (m_apb_pslverr),
    .m_apb_PREADY   (m_apb_pready),
    .m_apb_PWAKEUP  (m_apb_pwakeup),   // output: Requester wake-up
    .m_apb_PRUSER   (m_apb_pruser),
    .m_apb_PBUSER   (m_apb_pbuser),

    // Packed command interface
    .cmd_valid      (tb_cmd_valid),
    .cmd_ready      (tb_cmd_ready),
    .cmd_data       (tb_cmd_data),

    // Packed response interface
    .rsp_valid      (tb_rsp_valid),
    .rsp_ready      (tb_rsp_ready),
    .rsp_data       (tb_rsp_data),

    // Status
    .parity_error_rdata (),
    .parity_error_ctrl  ()
);
```

```
// Build command packet.
// Field widths follow the module parameters; the literals below
assume the
// defaults, giving CMD_PACKET_WIDTH = 82.
assign tb_cmd_data = {
    1'b1,           // last     (1 bit)
    1'b1,           // first    (1 bit)
    1'b1,           // pwrite   (1 bit, write transaction)
    3'b000,         // pprot    (PROT_WIDTH  = 3)
    4'hF,           // pstrb    (STRB_WIDTH  = 4, all bytes)
    32'h1000_0000,  // paddr    (ADDR_WIDTH  = 32)
    32'hDEAD_BEEF,  // pwdata   (DATA_WIDTH  = 32)
    4'h0,           // pauser   (AUSER_WIDTH = 4)
    4'h0            // pwuser   (WUSER_WIDTH = 4)
};
```

For a parameterized testbench, build the packet from the parameters rather than from literals so it tracks any width change:

```
assign tb_cmd_data = {
    cmd_last, cmd_first, cmd_pwrite,
    cmd_pprot[PROT_WIDTH-1:0],
    cmd_pstrb[STRB_WIDTH-1:0],
    cmd_paddr[ADDR_WIDTH-1:0],
    cmd_pwdata[DATA_WIDTH-1:0],
    cmd_pauser[AUSER_WIDTH-1:0],
    cmd_pwuser[WUSER_WIDTH-1:0]
};
```

4.7 Design Notes

4.7.1 First/Last Markers

The first/last bits in packets enable: - Transaction boundary detection - Burst transaction support - Testbench synchronization

4.7.2 Internal Architecture

The stub instantiates the full `apb5_master` internally: - All protocol handling done by proven `apb5_master` - Stub only handles packet packing/unpacking - Maintains full APB5 compliance

---

4.8 Related Modules

- **APB5 Master** - Core master module (wrapped by stub)
- **APB5 Slave Stub** - Corresponding slave stub

4.9 Testing

`val/amba/test_apb5_master_stub.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_apb5_master_stub.py -v
```

4.10 References

- Arm. *AMBA APB Protocol Specification*, ARM IHI 0024E, Feb. 2023 (APB5 first published in Issue D, Apr. 2021). developer.arm.com/documentation/ihi0024/e – ch. 3 “Transfers”, ch. 4 “Operating States”, §3.7 “Wake-up signaling” and §3.8 “User signaling”: the protocol of the wrapped APB5 block. Citation only.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – the valid/ready channel discipline behind the `gaxi_skid_buffer` queues: a buffered stage may stall or add latency without changing what the endpoints compute.
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`)

4.11 Navigation

- **← Back to APB5 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

5 APB5 Monitor

Module: `apb5_monitor.sv` **Location:** `rtl/amba/apb5/` (protocol monitors live with their protocol; only the monitor CORE pieces are in `rtl/amba/ocelus/`) **Status:** Production Ready

---

5.1 Overview

The APB5 Monitor provides comprehensive protocol monitoring for APB5 interfaces with support for all APB5 extensions. It tracks transactions, detects errors, monitors performance, and reports events through a standardized monitor bus interface.

5.1.1 Key Features

- Full APB5 protocol monitoring
- Wake-up event tracking (APB5 PWAKEUP)
- Parity error detection (when `ENABLE_PARITY_MON = 1`)
- Transaction timeout detection
- Performance latency measurement
- Protocol violation detection
- 128-bit monitor bus packet output with side-band timestamp (same format as `apb4_monitor`)

5.1.2 Not Implemented

The `cmd_pauser` / `cmd_pwuser` / `rsp_pruser` / `rsp_pbuser` ports and the `cfg_user_enable` input are accepted so the interface is stable for integrators, but **no user-signal logic exists** and no `APB5_USER_*` event code is ever produced. Earlier revisions of this page and of the module header claimed PAUSER/PWUSER/PRUSER/PBUSER tracking; that claim was incorrect. Emitting a user-signal event per transaction would flood the monitor bus, so the feature is deferred rather than half-wired.

5.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
AUSER_WIDTH	int	4	Address user signal width

Parameter	Type	Default	Description
WUSER_WIDTH	int	4	Write user signal width
RUSER_WIDTH	int	4	Read user signal width
BUSER_WIDTH	int	4	Response user signal width
UNIT_ID	logic [7:0]	8'h01	8-bit unit identifier
AGENT_ID	logic [15:0]	16'h000A	16-bit agent identifier
MAX_TRANSACTIONS	int	4	Maximum concurrent transactions
MONITOR_FIFO_DEPTH	int	8	Monitor packet FIFO depth
ENABLE_PARITY_MON	bit	0	Enable parity monitoring
N_ADDR_RANGES	int	0	Number of `apb_monitor_addr_check` comparators. 0 = address-range checker not synthesized
USE_MONITOR	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.

5.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
AW	ADDR_WIDTH
DW	DATA_WIDTH
SW	DW/8
AUW	AUSER_WIDTH

Derived parameter	Default expression
WUW	WUSER_WIDTH
RUW	RUSER_WIDTH
BUW	BUSER_WIDTH

5.3 Ports

5.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	Monitor clock
aresetn	1	Input	Monitor reset (active low)

5.3.2 Command Interface Monitoring

Port	Width	Direction	Description
cmd_valid	1	Input	Command valid signal
cmd_ready	1	Input	Command ready signal
cmd_pwrite	1	Input	Write/read indicator
cmd_paddr	ADDR_WIDTH	Input	Command address
cmd_pwdata	DATA_WIDTH	Input	Command write data
cmd_pstrb	STRB_WIDTH	Input	Write byte strobes
cmd_pprot	3	Input	Protection attributes
cmd_pauser	AUSER_WIDTH	Input	Address user signal
cmd_pwuser	WUSER_WIDTH	Input	Write user signal

5.3.3 Response Interface Monitoring

Port	Width	Direction	Description
rsp_valid	1	Input	Response valid signal
rsp_ready	1	Input	Response ready signal
rsp_prdata	DATA_WIDTH	Input	Response read data
rsp_pslverr	1	Input	Slave error response
rsp_pruser	RUSER_WIDTH	Input	Read user signal
rsp_pbuser	BUSER_WIDTH	Input	Response user signal

5.3.4 APB5 Extension Monitoring

Port	Width	Direction	Description
apb5_pwakeup	1	Input	APB5 wake-up signal
parity_error_wdata	1	Input	Write data parity error
parity_error_rdata	1	Input	Read data parity error
parity_error_ctrl	1	Input	Control parity error

5.3.5 Configuration Inputs

Port	Width	Direction	Description
cfg_error_enable	1	Input	Enable error reporting
cfg_timeout_enable	1	Input	Enable timeout detection
cfg_protocol_enable	1	Input	Enable protocol checking

Port	Width	Direction	Description
cfg_slverr_enable	1	Input	Enable SLVERR reporting
cfg_parity_enable	1	Input	Enable parity error reporting
cfg_wakeup_enable	1	Input	Enable wake-up event reporting
cfg_user_enable	1	Input	Enable user signal reporting
cfg_perf_enable	1	Input	Enable performance reporting
cfg_latency_enable	1	Input	Enable latency threshold
cfg_cmd_timeout_cnt	16	Input	Command timeout threshold
cfg_rsp_timeout_cnt	16	Input	Response timeout threshold
cfg_latency_threshold	32	Input	Latency threshold value
cfg_wakeup_timeout_cnt	16	Input	Wake-up timeout threshold

5.3.6 Address-Range Checker Configuration

Active only when `N_ADDR_RANGES > 0`; otherwise these inputs are ignored and the `apb_monitor_addr_check` block is not synthesized. Range-violation packets are merged onto the monitor bus at lower priority than the event FIFO.

Port	Width	Direction	Description
cfg_addr_c	1	Input	Master enable for the

Port	Width	Direction	Description
heck_enable			address-range checker
cfg_addr_range_enable	N_ADDR_RANGES	Input	Per-range enable bit vector
cfg_addr_range_low	N_ADDR_RANGES × ADDR_WIDTH	Input	Per-range low (inclusive) bounds
cfg_addr_range_high	N_ADDR_RANGES × ADDR_WIDTH	Input	Per-range high (inclusive) bounds

5.3.7 Monitor Bus Output

The APB5 monitor emits the standard **128-bit `monitor_packet_t`** plus the 64-bit side-band `monbus_timestamp`, identical to `apb4_monitor`. `i_mon_time` is the free-running monitor-time broadcast from the `monbus_group` family; FIFO events sample it at emission; address-range packets sample it the same way – `apb_monitor_addr_check` holds no timestamp register, so the value is the emission-cycle time, not a match-cycle latch.

Port	Width	Direction	Description
i_mon_time	64	Input	Free-running monitor-time broadcast
monbus_valid	1	Output	Monitor packet valid
monbus_ready	1	Input	Monitor bus ready
monbus_packet	128	Output	128-bit monitor packet (see format below)
monbus_timestamp	64	Output	Side-band time sampled at emission

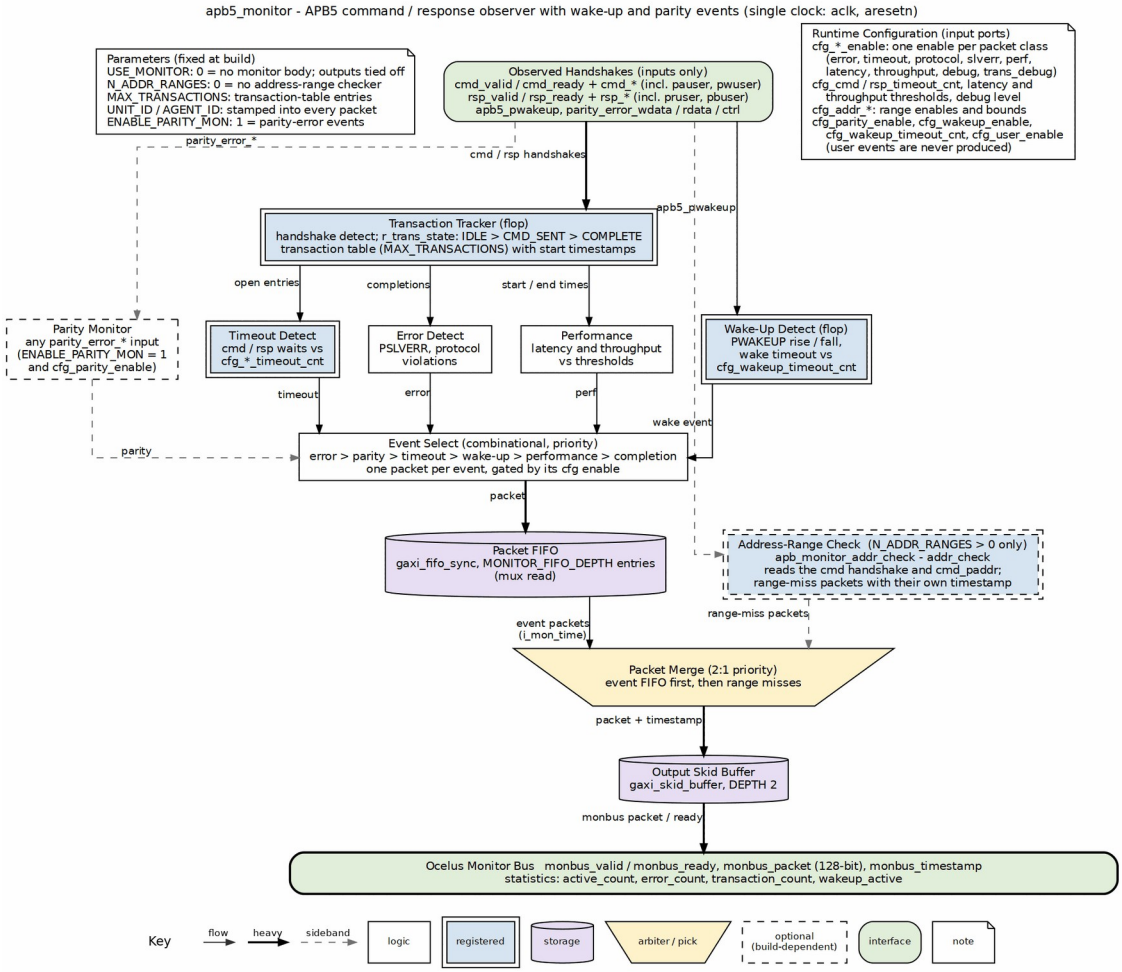
5.3.8 Status Outputs

Port	Width	Direction	Description
active_count	8	Output	Active

Port	Width	Direction	Description
			transaction count
error_count	16	Output	Total error count
transaction_count	32	Output	Total transaction count
wakeup_active	1	Output	Wake-up currently active

5.4 Functional Description

5.4.1 Block diagram



apb5_monitor block diagram

`apb4_monitor` plus the APB5 event sources. The transaction tracker, timeout, error and performance checks work as in `apb4_monitor`; a wake-up detector watches `apb5_pwakeup` (rise, fall and a wake timeout) and a parity monitor (`ENABLE_PARITY_MON = 1`) watches the `parity_error_*` inputs. The event select emits one packet per event in priority order error > parity > timeout > wake-up > performance > completion. The address-range checker is a separate packet source that a 2:1 priority merge (event FIFO first) combines with the FIFO stream ahead of the output skid. User-signal events are never produced.

5.4.2 Monitor Packet Format

5.4.2.1 128-bit Packet Structure

The APB5 monitor drives the standard 128-bit packet built by `ocelus_common_pkg::create_monitor_packet`, paired with the 64-bit side-band `monbus_timestamp`. See monitor_package_spec.md for the canonical definition.

```
Bits [127:124] - Packet Type (4 bits, see table below)
Bits [123:109] - Reserved (15 bits)
Bits [108:105] - Protocol (4 bits): fixed to PROTOCOL_APB (0x2)
Bits [104: 97] - Event Code (8 bits, APB/APB5-specific)
Bits [ 96: 88] - Channel ID (9 bits, always 0 for APB)
Bits [ 87: 72] - Agent ID (16 bits, from AGENT_ID)
Bits [ 71: 64] - Unit ID (8 bits, from UNIT_ID)
Bits [ 63:  0] - Event Data (64 bits)
```

For FIFO-sourced events the event-data field is `{24'h0, aux_data[7:0], event_data[31:0]}`. For address-range violations from `apb_monitor_addr_check` it is `{range_index[3:0], is_read, addr[58:0]}`.

> **History:** before the fix for issue #41 this module connected the 128-bit `addr_pkt_data` output of `apb_monitor_addr_check` to a 64-bit net. Every header field was discarded and `event_data` was re-decoded against the legacy 64-bit layout, so a range-3 violation surfaced as `packet_type = 0x3` (Timeout) with `protocol = AXI`, `event_code = 0x00`, and the address shifted right by 3. The path was uncovered because every harness — including the formal proof — left `N_ADDR_RANGES` at its default of 0, which elides the checker entirely.

5.4.3 Packet Types

Packet-type codes follow `ocelus_common_pkg`. The APB5 monitor emits the subset marked below; other codes are defined for cross-protocol consistency.

Value	Type	Emitted by APB5 monitor	Description
0x0	Error		SLVERR, protocol violations, parity errors
0x1	Completion		Transaction completed without error
0x2	Threshold	—	Threshold-crossed events (not

Value	Type	Emitted by APB5 monitor	Description
			generated here)
0x3	Timeout		Command / response / wake-up timeout
0x4	Performance		Latency-threshold-exceeded metric
0x8	AddrMatch	—	Not generated. Address-range violations are emitted as **Error** (`0x0`) packets with event code `APB_ERR_ADDR_RANGE` (`0x08`)
0x9	APB		APB-specific events (wake-up request/acknowledge)
0xF	Debug	—	Debug/trace events (not generated here)

Note: parity error events are emitted as **Error** packets (type `0x0`) with an APB5 parity event code — not as a distinct packet type.

5.4.4 Event Edge Detection

Timeout, protocol-violation, parity and latency-threshold conditions are all *levels* — once true they stay true until the underlying condition clears. Every one of them is edge-qualified before it can write the event FIFO, so a single occurrence produces exactly one packet. Without this, one stuck command emitted an identical timeout packet on every cycle the condition held (29 packets over a 40-cycle stall in the regression test). Regression coverage: `val/amba/test_apb5_monitor.py::test_apb5_monitor_timeout_edge`.

5.4.5 APB5-Specific Event Codes

Event codes live in per-category enums, so a given numeric code is disambiguated by the packet type it rides on. Wake-up codes ride the **APB** packet type (`0x9`); parity codes ride the **Error** packet type (`0x0`).

Event	Code	Packet Type	Description
APB5_WAKEUP_REQUEST	0x0	APB (0x9)	PWAKEUP rising edge
APB5_WAKEUP_ACKNOWLEDGED	0x1	APB (0x9)	PWAKEUP falling edge
APB5_PARITY_PWDATA_ERROR	0x0	Error (0x0)	Write data parity error
APB5_PARITY_PRDATA_ERROR	0x1	Error (0x0)	Read data parity error
APB5_PARITY_PREADY_ERROR	0x2	Error (0x0)	PREADY parity error

```
stateDiagram-v2
    [*] --> IDLE

    IDLE --> CMD_SENT : cmd_valid & cmd_ready
    CMD_SENT --> COMPLETE : rsp_valid & rsp_ready
    COMPLETE --> IDLE : (next cycle)

    state IDLE {
        note right of IDLE : No active transaction
    }
    state CMD_SENT {
        note right of CMD_SENT : Waiting for response
    }
    state COMPLETE {
        note right of COMPLETE : Transaction finished
    }
```

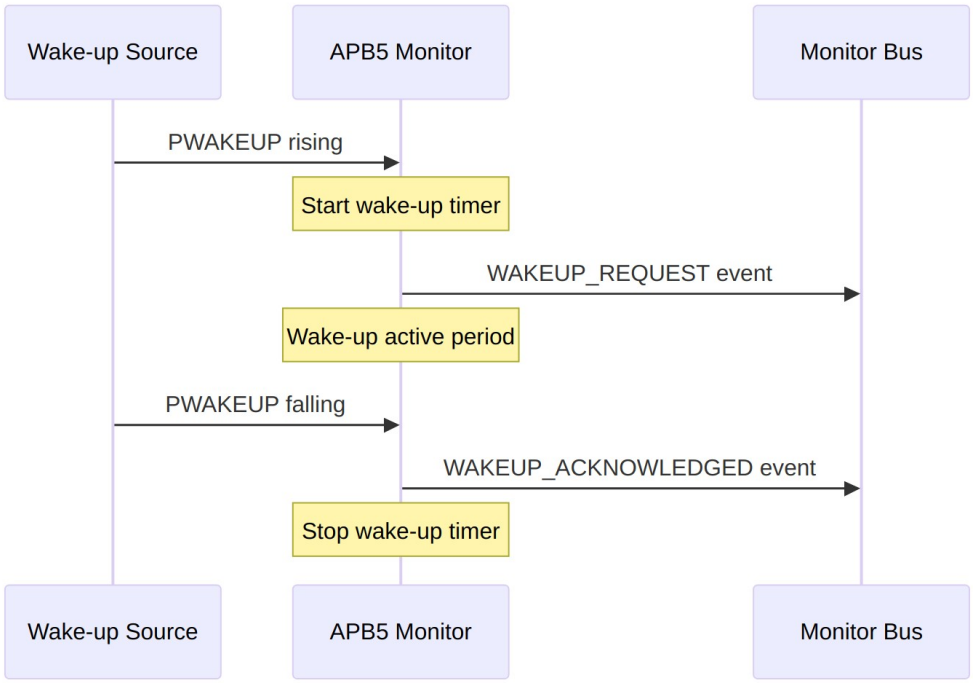
5.4.6 Event Priority

Events are generated with the following priority (highest first):

1. **Error Events** - Protocol violations, SLVERR
2. **Parity Events** - Parity errors detected
3. **Timeout Events** - Command/response timeouts
4. **Wake-up Events** - PWAKEUP transitions

5. **Performance Events** - Latency threshold exceeded
6. **Completion Events** - Normal transaction completion

5.4.7 Wake-up Monitoring



Wake-up Detection

5.4.7.1 Wake-up Timeout

If PWAKEUP remains high longer than `cfg_wakeup_timeout_cnt`, a timeout event is generated.

5.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

5.6 Usage Examples

```
apb5_monitor #(
    .ADDR_WIDTH         (32),
    .DATA_WIDTH         (32),
    .AUSER_WIDTH        (4),
    .WUSER_WIDTH        (4),
    .RUSER_WIDTH        (4),
    .BUSER_WIDTH        (4),
    .UNIT_ID            (1),
    .AGENT_ID           (10),
    .MAX_TRANSACTIONS   (4),
    .MONITOR_FIFO_DEPTH (8),
    .ENABLE_PARITY_MON  (0)
) u_apb5_monitor (
    .aclk               (apb_clk),
    .aresetn            (apb_rst_n),

    // Command interface monitoring
    .cmd_valid          (apb_cmd_valid),
    .cmd_ready          (apb_cmd_ready),
    .cmd_pwrite         (apb_cmd_pwrite),
    .cmd_paddr          (apb_cmd_paddr),
    .cmd_pwdata         (apb_cmd_pwdata),
    .cmd_pstrb          (apb_cmd_pstrb),
    .cmd_pprot          (apb_cmd_pprot),
    .cmd_pauser         (apb_cmd_pauser),
    .cmd_pwuser         (apb_cmd_pwuser),

    // Response interface monitoring
    .rsp_valid          (apb_rsp_valid),
    .rsp_ready          (apb_rsp_ready),
    .rsp_prdata         (apb_rsp_prdata),
    .rsp_pslverr        (apb_rsp_pslverr),
    .rsp_pruser         (apb_rsp_pruser),
    .rsp_pbuser         (apb_rsp_pbuser),

    // APB5 wake-up monitoring
    .apb5_pwakeup       (apb_pwakeup),

    // Parity error inputs
    .parity_error_wdata (1'b0),
    .parity_error_rdata (1'b0),
    .parity_error_ctrl  (1'b0),

    // Configuration
    .cfg_error_enable   (1'b1),
```

```
    .cfg_timeout_enable (1'b1),
    .cfg_protocol_enable(1'b1),
    .cfg_slverr_enable  (1'b1),
    .cfg_parity_enable  (1'b0),
    .cfg_wakeup_enable  (1'b1),
    .cfg_user_enable    (1'b0),
    .cfg_perf_enable    (1'b0),
    .cfg_latency_enable (1'b0),
    .cfg_cmd_timeout_cnt(16'd1000),
    .cfg_rsp_timeout_cnt(16'd1000),
    .cfg_latency_threshold(32'd100),
    .cfg_wakeup_timeout_cnt(16'd500),

    // Address-range checker (only when N_ADDR_RANGES > 0)
    .cfg_addr_check_enable (1'b0),
    .cfg_addr_range_enable ('0),
    .cfg_addr_range_low    ('0),
    .cfg_addr_range_high   ('0),

    // Monitor time broadcast (from the monbus_group family)
    .i_mon_time         (mon_time),

    // Monitor bus output (128-bit packet + 64-bit side-band
timestamp)
    .monbus_valid       (apb_mon_valid),
    .monbus_ready       (apb_mon_ready),
    .monbus_packet      (apb_mon_packet),
    .monbus_timestamp   (apb_mon_timestamp),

    // Status
    .active_count       (apb_active_count),
    .error_count        (apb_error_count),
    .transaction_count  (apb_trans_count),
    .wakeup_active      (apb_wakeup_active)
);
```

5.7 Design Notes

5.7.1 Transaction Table

- Tracks up to `MAX_TRANSACTIONS` concurrent transactions
- APB typically has 1-4 outstanding (simple protocol)
- Table entries cleaned up after event reported

5.7.2 Internal FIFOs

- Monitor FIFO depth configurable via `MONITOR_FIFO_DEPTH`
- Output skid buffer ensures no backpressure stalls

5.7.3 Configuration Guidelines

Functional Debug:

```
.cfg_error_enable   (1'b1),
.cfg_timeout_enable (1'b1),
.cfg_protocol_enable(1'b1),
.cfg_slverr_enable  (1'b1),
.cfg_wakeup_enable  (1'b1),
.cfg_perf_enable    (1'b0)   // Disable for lower traffic
```

Performance Analysis:

```
.cfg_error_enable   (1'b1),
.cfg_perf_enable    (1'b1),
.cfg_latency_enable (1'b1),
.cfg_wakeup_enable  (1'b0)   // Disable non-essential
```

5.8 Related Modules

- **APB5 Master** - APB5 master interface
- **APB5 Slave** - APB5 slave interface
- **Monitor Packet Format** - Standard packet format

5.9 Testing

`val/amba/test_apb5_monitor.py` exercises this module. It collects 5 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_apb5_monitor.py -v
```

5.10 References

- Arm. *AMBA APB Protocol Specification*, ARM IHI 0024E, Feb. 2023 (APB5 first published in Issue D, Apr. 2021). developer.arm.com/documentation/ihi0024/e — §3.4 “Error response” (`PSLVERR`, reported as error events), §3.7 “Wake-up signaling” and ch. 5

"Interface parity protection"; the transaction timeouts are repo monitoring policy with no spec or paper behind them. Citation only.

- Indexed in `rtl/amba/references/README.md`

5.11 Navigation

- **← Back to APB5 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

6 APB5 Slave

Module: `apb5_slave.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

6.1 Overview

The APB5 Slave module implements a complete AMBA APB5 slave interface with all APB5 extensions. It receives APB transactions from the bus and provides a command/response interface for backend logic, supporting user-defined signals, the Requester's wake-up input, and optional parity.

6.1.1 Key Features

- Full AMBA APB5 protocol compliance
- Receives PAUSER/PWUSER from master
- Generates PRUSER/PBUSER responses
- PWAKEUP input from the Requester (IHI 0024E §3.7.2), with PWAKEUPCHK checking
- Optional IHI 0024E ch. 5 interface parity (`*CHK` signals, odd parity)
- Command/response FIFO buffering
- Configurable buffer depth

6.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address/request user signal width
WUSER_WIDTH	int	4	Write data user signal width
RUSER_WIDTH	int	4	Read data user signal width
BUSER_WIDTH	int	4	Response user signal width

Parameter	Type	Default	Description
DEPTH	int	2	Skid-buffer depth in entries; must be 2..8 inclusive (any integer)
ENABLE_PARITY	bit	0	Enable parity generation and checking
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)

`DEPTH` sets both the command and the response `gaxi_skid_buffer` depth. It is a literal entry count, not a log2 exponent.

6.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
DW	DATA_WIDTH
AW	ADDR_WIDTH
SW	STRB_WIDTH
PW	PROT_WIDTH
AUW	AUSER_WIDTH
WUW	WUSER_WIDTH
RUW	RUSER_WIDTH
BUW	BUSER_WIDTH
CPW	AW + DW + SW + PW + AUW + WUW + 1
RPW	DW + RUW + BUW + 1

6.3 Ports

6.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low

Port	Width	Direction	Description
			reset

6.3.2 APB5 Slave Interface

Port	Width	Direction	Description
s_apb_PSEL	1	Input	APB select signal
s_apb_PENABLE	1	Input	APB enable signal
s_apb_PREADY	1	Output	Slave ready response
s_apb_PADDR	ADDR_WIDTH	Input	APB address
s_apb_PWRITE	1	Input	Write/read indicator
s_apb_PWDATA	DATA_WIDTH	Input	Write data
s_apb_PSTRB	STRB_WIDTH	Input	Write byte strobes
s_apb_PPROT	PROT_WIDTH	Input	Protection attributes
s_apb_PAUSER	AUSER_WIDTH	Input	User-defined request attributes
s_apb_PWUSER	WUSER_WIDTH	Input	User-defined write data attributes
s_apb_PRDATA	DATA_WIDTH	Output	Read data to master
s_apb_PSLVERR	1	Output	Slave error response
s_apb_PWAKEUP	1	Input	Requester wake-up (IHI 0024E §3.7.2); checked by PWAKEUPCHK, not waited on before PREADY

Port	Width	Direction	Description
s_apb_PRUSER	RUSER_WIDTH	Output	User-defined read data attributes
s_apb_PBUSER	BUSER_WIDTH	Output	User-defined response attributes

6.3.3 Parity Signals (Optional)

Port	Width	Direction	Description
s_apb_PADDRCHK	ceil(ADDR_WIDTH/8)	Input	Odd parity over PADDR, one bit per 8 bits
s_apb_PCTRLCHK	1	Input	Odd parity over PPROT and PWRITE (PNSE not carried)
s_apb_PSELCHK	1	Input	Odd parity over PSEL (the spec's PSELxCHK)
s_apb_PENABLECHK	1	Input	Odd parity over PENABLE
s_apb_PWDATACHK	DATA_WIDTH/8	Input	Odd parity over PWDATA, one bit per byte lane
s_apb_PSTRBCHK	1	Input	Odd parity over all of PSTRB
s_apb_PAUSERCHK	ceil(AUSER_WIDTH/8)	Input	Odd parity over PAUSER, one bit per 8 bits
s_apb_PWUSERCHK	ceil(WUSER_WIDTH/8)	Input	Odd parity over PWUSER, one bit per 8 bits
s_apb_PRDATACHK	DATA_WIDTH/8	Output	Odd parity over PRDATA, one bit per byte lane
s_apb_PREADYCHK	1	Output	Odd parity over PREADY
s_apb_PSLVERRCHK	1	Output	Odd parity over PSLVERR
s_apb_PWAKEUPCHK	1	Input	Odd parity over PWAKEUP
s_apb_PRU	ceil(RUSER_	Output	Odd parity over PRUSER,

Port	Width	Direction	Description
SERCHK	WIDTH/8)		one bit per 8 bits
s_apb_PBUSERCHK	ceil(BUSER_WIDTH/8)	Output	Odd parity over PBUSER, one bit per 8 bits

6.3.4 Command Interface (to backend)

Port	Width	Direction	Description
cmd_valid	1	Output	Command valid to backend
cmd_ready	1	Input	Backend ready
cmd_pwrite	1	Output	Command write/read
cmd_paddr	ADDR_WIDTH	Output	Command address
cmd_pwdata	DATA_WIDTH	Output	Command write data
cmd_pstrb	STRB_WIDTH	Output	Command write strobes
cmd_pprot	PROT_WIDTH	Output	Command protection
cmd_pauser	AUSER_WIDTH	Output	Command address user
cmd_pwuser	WUSER_WIDTH	Output	Command write user

6.3.5 Response Interface (from backend)

Port	Width	Direction	Description
rsp_valid	1	Input	Response valid from backend
rsp_ready	1	Output	Slave ready for response
rsp_prdata	DATA_WIDTH	Input	Response read data
rsp_pslverr	1	Input	Response error

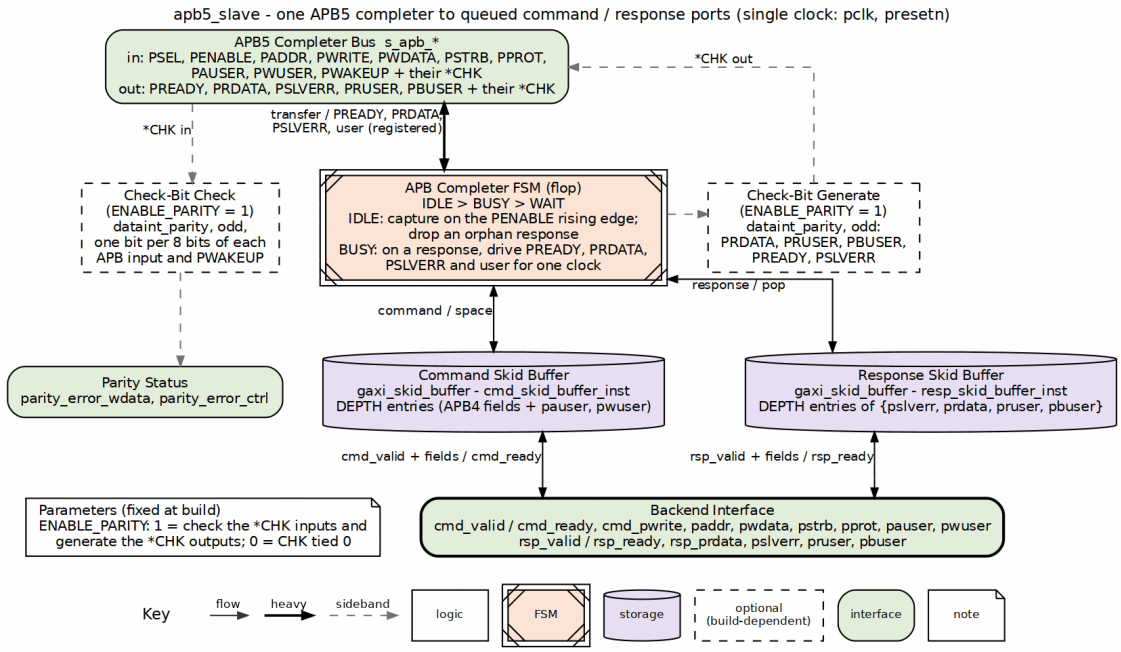
Port	Width	Direction	Description
			status
rsp_pruser	RUSER_WIDTH	Input	Response read user
rsp_pbuser	BUSER_WIDTH	Input	Response user

6.3.6 Status Outputs

Port	Width	Direction	Description
parity_error_wdata	1	Output	PWDATACHK, PSTRBCHK or PWUSERCHK mismatch (tied to 0 when ENABLE_PARITY=0)
parity_error_ctrl	1	Output	PADDRCHK, PCTRLCHK, PSELCHK, PENABLECHK, PAUSERCHK or PWAKEUPCHK mismatch (tied to 0 when ENABLE_PARITY=0)

6.4 Functional Description

6.4.1 Block diagram



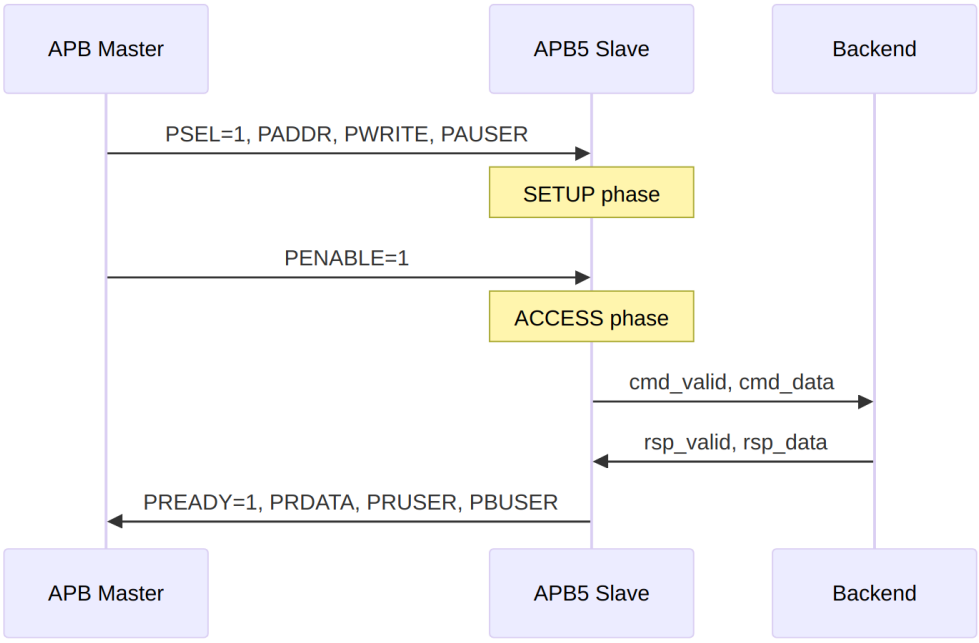
apb5_slave block diagram

Same transfer / response lanes as `apb4_slave`, with the user fields carried through both skids. With `ENABLE_PARITY = 1`, the check-bit check covers every APB input (and PWAKEUP) and reports `parity_error_wdata` / `parity_error_ctrl`, while the check-bit generate block drives the *CHK outputs for PRDATA, PRUSER, PBUSER, PREADY and PSLVERR. PWAKEUP itself is only checked here; this completer does not wait for it.

6.4.2 APB5 Slave Protocol

The slave responds to APB transactions with proper timing:

1. **SETUP Phase**: PSEL=1, PENABLE=0 - Capture address and control
2. **ACCESS Phase**: PSEL=1, PENABLE=1 - Assert PREADY when response ready



Response Timing

6.5 Timing Characteristics

6.5.1 Write Transaction with Wait States

Timing diagram pending. The signals and sequence this scenario exercises:

- PCLK
- PSEL
- PENABLE
- PADDR
- PWRITE (high)
- PWDATA
- PAUSER, PWUSER
- PREADY (with wait states)
- PSLVERR
- PBUSER

6.5.2 Read Transaction

Timing diagram pending. The signals and sequence this scenario exercises:

- PCLK
- PSEL
- PENABLE
- PADDR
- PWRITE (low)
- PAUSER
- PREADY
- PRDATA
- PRUSER, PBUSER

6.5.3 Wake-up Signaling

Timing diagram pending. The signals and sequence this scenario exercises:

- PCLK
- PWAKEUP (from the master) rising ahead of PSEL
- PSEL/PENABLE/PREADY with PWAKEUP held through PREADY

6.6 Usage Examples

```
apb5_slave #(
    .ADDR_WIDTH     (32),
    .DATA_WIDTH     (32),
    .AUSER_WIDTH    (4),
    .WUSER_WIDTH    (4),
    .RUSER_WIDTH    (4),
    .BUSER_WIDTH    (4),
    .DEPTH          (2),
    .ENABLE_PARITY  (0)
) u_apb5_slave (
    .pclk           (apb_clk),
    .presetn        (apb_rst_n),

    // APB5 slave interface
    .s_apb_PSEL     (s_apb_psel),
    .s_apb_PENABLE  (s_apb_penable),
    .s_apb_PREADY   (s_apb_pready),
```

```
    .s_apb_PADDR    (s_apb_paddr),
    .s_apb_PWRITE   (s_apb_pwrite),
    .s_apb_PWDATA   (s_apb_pwdata),
    .s_apb_PSTRB    (s_apb_pstrb),
    .s_apb_PPROT    (s_apb_pprot),
    .s_apb_PAUSER   (s_apb_pauser),
    .s_apb_PWUSER   (s_apb_pwuser),
    .s_apb_PRDATA   (s_apb_prdata),
    .s_apb_PSLVERR  (s_apb_pslverr),
    .s_apb_PWAKEUP  (s_apb_pwakeup),   // input: from the Requester
    .s_apb_PRUSER   (s_apb_pruser),
    .s_apb_PBUSER   (s_apb_pbuser),

    // Backend command interface
    .cmd_valid      (backend_cmd_valid),
    .cmd_ready      (backend_cmd_ready),
    .cmd_pwrite     (backend_cmd_write),
    .cmd_paddr      (backend_cmd_addr),
    .cmd_pwdata     (backend_cmd_wdata),
    .cmd_pstrb      (backend_cmd_strb),
    .cmd_pprot      (backend_cmd_prot),
    .cmd_pauser     (backend_cmd_auser),
    .cmd_pwuser     (backend_cmd_wuser),

    // Backend response interface
    .rsp_valid      (backend_rsp_valid),
    .rsp_ready      (backend_rsp_ready),
    .rsp_prdata     (backend_rsp_rdata),
    .rsp_pslverr    (backend_rsp_error),
    .rsp_pruser     (backend_rsp_ruser),
    .rsp_pbuser     (backend_rsp_buser),


    // Parity interface (unused here because ENABLE_PARITY=0)
    .s_apb_PADDRCHK      ('0),
    .s_apb_PCTRLCHK      (1'b0),
    .s_apb_PSELCHK       (1'b0),
    .s_apb_PENABLECHK    (1'b0),
    .s_apb_PWDATACHK     ('0),
    .s_apb_PSTRBCHK      (1'b0),
    .s_apb_PAUSERCHK     ('0),
    .s_apb_PWUSERCHK     ('0),
    .s_apb_PRDATACHK     (),
    .s_apb_PREADYCHK     (),
    .s_apb_PSLVERRCHK    (),
    .s_apb_PWAKEUPCHK    (1'b0),
    .s_apb_PRUSERCHK     (),
```

```
    .s_apb_PBUSERCHK     (),
    .parity_error_wdata  (),
    .parity_error_ctrl   ()
);
```

6.7 Design Notes

6.7.1 Backpressure Handling

- If backend cannot accept commands (`cmd_ready=0`), slave inserts wait states
- PREADY held low until backend accepts command and provides response

6.7.2 User Signal Propagation

- PAUSER/PWUSER captured during SETUP phase, forwarded to backend
- PRUSER/PBUSER from backend driven during ACCESS phase response

6.7.3 Wake-up Input

`s_apb_PWAKEUP` is an input: IHI 0024E §3.7.2 (Table 3-7) makes PWAKEUP a Requester signal. A Completer is permitted to wait for it before PREADY; this one does not, so a Requester that never asserts PWAKEUP cannot deadlock it. Inside `apb5_slave` PWAKEUP feeds only the `PWAKEUPCHK` parity check; the `_cg` wrappers also use it as a clock-gate wake term. There is no longer a backend `wakeup_request` port – a Completer has no APB wake-up output.

6.7.4 Parity Implementation

When `ENABLE_PARITY=1` the slave checks the requester check signals and drives the completer ones (`PRDATACHK` to `PBUSERCHK` below). With `ENABLE_PARITY=0` the outputs are tied LOW and the inputs are ignored.

Parity follows AMBA APB IHI 0024E chapter 5 (§5.3, Table 5-1):

- **Odd parity.** A payload slice plus its check bit always hold an odd number of ones. On a single-bit critical control the check bit is therefore the inversion of the signal.
- **One check bit per 8 payload bits.** Bit `[n]` of a multi-bit check signal covers payload bits `[8n+7:8n]`; when the payload is not a whole number of bytes, the top check bit covers the bits that remain. `PADDRCHK` on a 12-bit address is therefore 2 bits: `[7:0]` and `[11:8]`.

- **Coverage sets.** `PCTRLCHK` covers `PPROT` and `PWRITE` (and `PNSE`, which this module does not carry, so it counts as LOW). `PSTRB` has its own single `PSTRBCHK`.
- Each check bit is computed by an `rtl/common/dataint_parity` instance with `CHUNKS = 1` and `parity_type = 0` (odd), one instance per spec byte, so the spec's short final byte comes out at any width.

Check signal	Covers	Width	Check Enable (Table 5-1)
PADDRCHK	PADDR	ceil(ADDR_WIDTH/8)	PSEL
PCTRLCHK	PPROT, PWRITE	1	PSEL
PSELCHK	PSEL (spec PSELxCHK)	1	PRESETn
PENABLECHK	PENABLE	1	PSEL
PWDATACHK	PWDATA	DATA_WIDTH/8	PSEL & PWRITE
PSTRBCHK	PSTRB	1	PSEL & PWRITE
PAUSERCHK	PAUSER	ceil(AUSER_WIDTH/8)	PSEL
PWUSERCHK	PWUSER	ceil(WUSER_WIDTH/8)	PSEL & PWRITE
PRDATACHK	PRDATA	DATA_WIDTH/8	PSEL & PENABLE & PREADY & !PWRITE
PREADYCHK	PREADY	1	PSEL & PENABLE
PSLVERRCHK	PSLVERR	1	PSEL & PENABLE & PREADY
PWAKEUPCHK	PWAKEUP	1	PRESETn
PRUSERCHK	PRUSER	ceil(RUS	PSEL & PENABLE & PREADY & !

Check signal	Covers	Width	Check Enable (Table 5-1)
		ER_WIDTH/8)	PWRITE
PBUSERCHK	PBUSER	ceil(BUSER_WIDTH/8)	PSEL & PENABLE & PREADY

PWAKEUPCHK travels with PWAKEUP: both are Requester outputs (IHI 0024E §3.7.2, Table 3-7), generated by the master and checked by the slave.

The error flags are combinational and assert only while the failing check's Check Enable term holds:

- `parity_error_wdata`: PWDATACHK, PSTRBCHK or PWUSERCHK wrong during a write (PSEL & PWRITE).
- `parity_error_ctrl`: PADDRCHK, PCTRLCHK, PENABLECHK or PAUSERCHK wrong while PSEL, or PSELCHK or PWAKEUPCHK wrong at any time out of reset.

The slave reports mismatches but does not turn them into PSLVERR; the spec leaves that policy to the system (§5.4).

6.8 Related Modules

- **APB5 Master** - APB5 master interface
- **APB5 Slave CG** - Clock-gated variant
- **APB5 Slave CDC** - Clock domain crossing variant
- **APB4 Slave** - APB4 version for comparison

6.9 Testing

`val/amba/test_apb5_slave.py` exercises this module. It collects 5 parameter cases at the default REG_LEVEL; the two ENABLE_PARITY=1 cases also run `cocotb_test_apb5_slave_parity`, a directed IHI 0024E ch. 5 check (generated *CHK values, flag-per-check error injection, Check Enable gating). The `formal/amba/apb5_slave` proof checks the same parity rules with ENABLE_PARITY=1.

```
source env_python
pytest val/amba/test_apb5_slave.py -v
```

6.10 References

- Arm. *AMBA APB Protocol Specification*, ARM IHI 0024E, Feb. 2023 (APB5 first published in Issue D, Apr. 2021). developer.arm.com/documentation/ihi0024/e – §3.7 “Wake-up signaling” (`PWAKEUP`), §3.8 “User signaling” (`PAUSER`/`PWUSER`/`PRUSER`/`PBUSER`), ch. 5 “Interface parity protection” (`ENABLE_PARITY`); transfers and operating states (ch. 3-4) are as in APB4. The `*CHK` signals follow §5.3 and Table 5-1: odd parity, one check bit per 8 payload bits, `PCTRLCHK` over `PPROT`/`PWRITE` and a separate `PSTRBCHK` (issue #102 item 1). Citation only.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – the valid/ready channel discipline behind the `gaxi_skid_buffer` queues: a buffered stage may stall or add latency without changing what the endpoints compute.
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects”, section “ARM AMBA interconnect protocols suite” (APB bus protocol overview) – textbook treatment (secondary source).
- Chipressian. *Crack the Hardware Interview - from RTL Designers’ Perspective*, Part 2 ch. 2 “Handshake Protocols” (valid-ready protocol and valid-ready slices) – textbook treatment of the skid buffer (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`)

6.11 Navigation

- **← Back to APB5 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

7 APB5 Slave CDC

Module: `apb5_slave_cdc.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

7.1 Overview

The APB5 Slave CDC module provides clock domain crossing between an APB5 bus clock domain and a backend clock domain: full APB5 slave on the `pclk` side, a command/response backend interface on the `aclk` side, and transactions carried safely across the asynchronous boundary in between.

7.1.1 Key Features

- Full APB5 protocol support with CDC
- Asynchronous FIFO-based clock domain crossing (Gray pointers by default; Johnson available via `USE_JOHNSON`, and opt-in by design)
- All APB5 user signals (PAUSER, PWUSER, PRUSER, PBUSER)
- PWAKEUP is a `pclk`-domain input from the Requester (IHI 0024E §3.7.2)
- Single `DEPTH` parameter sizes both directions
- Metastability protection with 2-flop pointer synchronizers

7.2 Parameters

Parameter	Type	Default	Description
`ADDR_WIDTH`	int	32	APB address bus width
`DATA_WIDTH`	int	32	APB data bus width
`STRB_WIDTH`	int	DATA_WIDTH/8	Write strobe width (calculated)
`PROT_WIDTH`	int	3	Protection signal width
`AUSER_WIDTH`	int	4	Address user signal width
`WUSER_WIDTH`	int	4	Write user signal width
`RUSER_WIDTH`	int	4	Read user signal width
`BUSER_WIDTH`	int	4	Response user signal width
`DEPTH`	int	2	Skid-buffer depth of the

Parameter	Type	Default	Description
			wrapped `apb5_slave`; 2..8 inclusive (any integer)
`USE_JOHNSON`	int	0 (Gray)	CDC-FIFO pointer encoding: `0` Gray, `1` Johnson, `-1` auto-select. See below.
`ENABLE_PARITY`	bit	0	Enable parity generation and checking
`USE_2_PHASE_CDC`	bit	1	Deprecated and ignored — retained for source compatibility

There is no `CMD_DEPTH`, `RSP_DEPTH` or `SYNC_STAGES` parameter on this module. The two asynchronous FIFOs are sized internally from `DEPTH`:

```
localparam int CDC_FIFO_DEPTH = (DEPTH < 4) ? 4 : DEPTH;
```

so the CDC FIFOs are never shallower than 4 entries regardless of `DEPTH`. The synchronizer depth is fixed at 2 flops (`N_FLOP_CROSS(2)` on both `gaxi_fifo_async` instances) and is not exposed as a parameter. If your design needs 3-flop synchronization for an extreme clock ratio, you're editing the instantiation in the RTL.

7.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`DW`	`DATA_WIDTH`
`AW`	`ADDR_WIDTH`
`SW`	`STRB_WIDTH`
`PW`	`PROT_WIDTH`
`AUW`	`AUSER_WIDTH`
`WUW`	`WUSER_WIDTH`
`RUW`	`RUSER_WIDTH`
`BUW`	`BUSER_WIDTH`
`CPW`	`AW + DW + SW + PW + AUW +`

Derived parameter	Default expression
	`WUW + 1`
`RPW`	`DW + RUW + BUW + 1`

7.3 Ports

7.3.1 Clock and Reset

Port	Width	Direction	Description
`pclk`	1	Input	APB bus clock
`presetn`	1	Input	APB reset (active low)
`aclk`	1	Input	Backend/user clock
`aresetn`	1	Input	Backend/user reset (active low)

The backend clock and reset are named `aclk` and `aresetn`, matching the rest of the AMBA library. There are no `bclk`/`bresetn` ports.

7.3.2 APB5 Slave Interface

Same as apb5_slave - operates in `pclk` domain, including the optional parity signals.

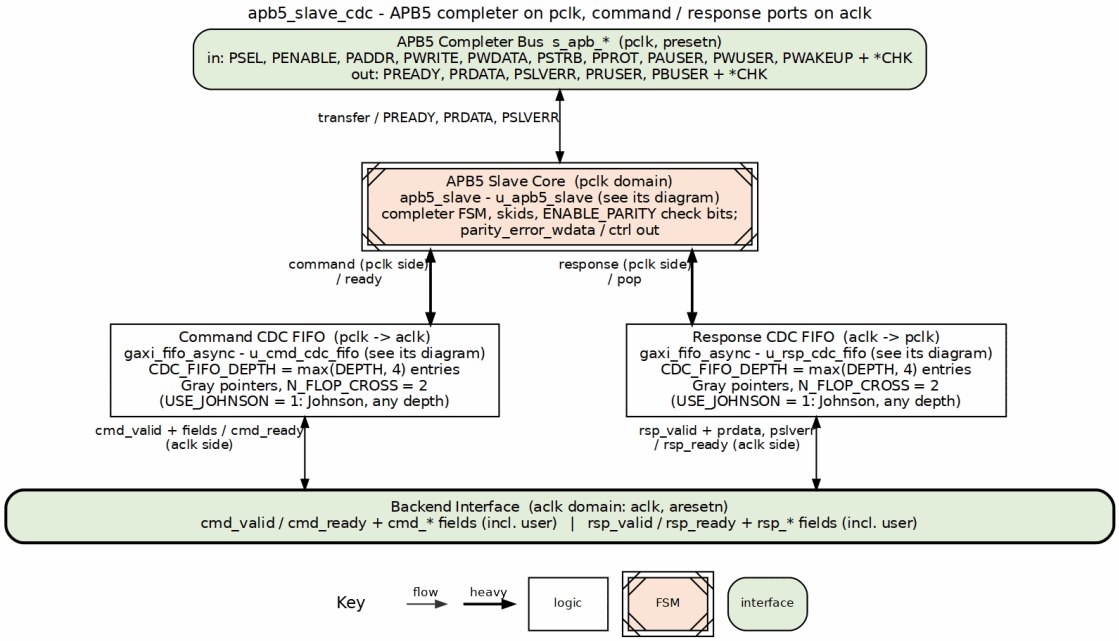
7.3.3 Backend Interface

Same command/response interface as apb5_slave - operates in the `aclk` domain. There is no backend `wakeup_request` input: PWAKEUP is a Requester signal (IHI 0024E §3.7.2) and arrives on the `pclk`-side APB port.

`parity_error_wdata` and `parity_error_ctrl` are **not** `aclk`-domain signals, despite sitting next to these ports. `apb5_slave` drives them combinationally from the APB inputs, qualified by each check's IHI 0024E Table 5-1 Check Enable term (`PSEL`, `PSEL & PWRITE`, or reset deasserted for `PSELCHK`; see apb5_slave), so they are `pclk`-domain (gated-`pclk` in the CG variant) and they cross no synchronizer. Sample them in `pclk`, or synchronize them yourself before using them in `aclk`.

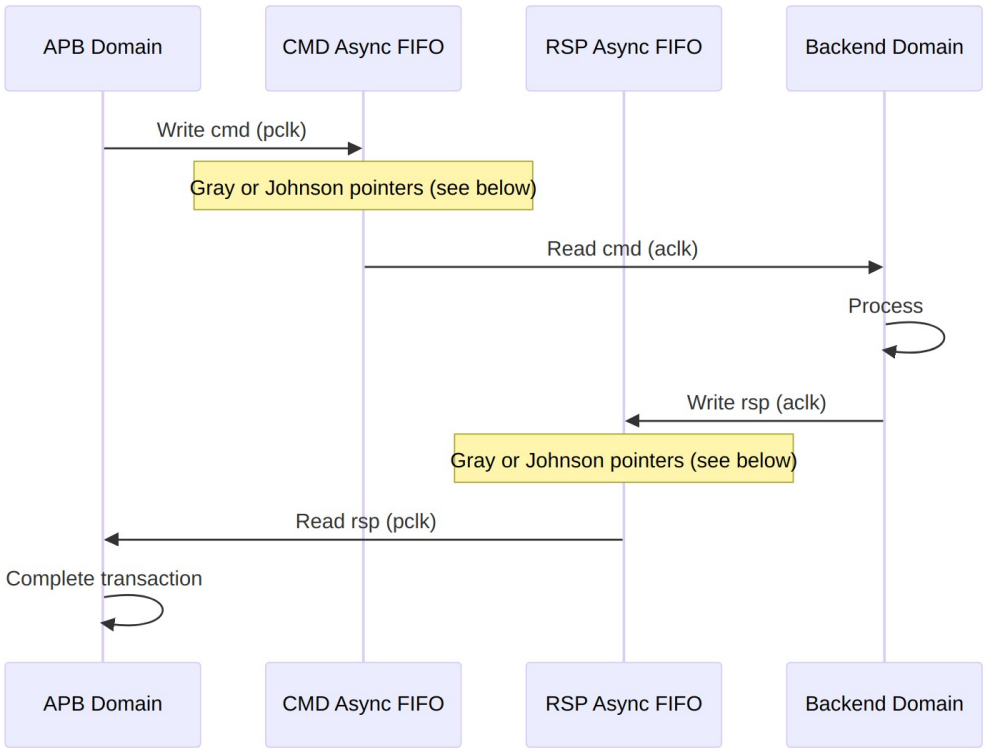
7.4 Functional Description

7.4.1 Architecture



apb5_slave_cdc block diagram

The APB side runs on `pclk`: the bus feeds the wrapped `apb5_slave` (one box; see its diagram). Each direction crosses to the `aclk` backend through its own `gaxi_fifo_async` (`max(DEPTH, 4)` entries): the command drops down the left lane (pclk to aclk), the response climbs the right lane (aclk to pclk). The Gray-pointer crossings are inside the async FIFOs.



CDC Mechanism

Both directions cross through `gaxi_fifo_async` instances with a fixed 2-flop synchronizer (`N_FLOP_CROSS(2)`) on each pointer crossing. The command FIFO is written in `pclk` and read in `aclk`; the response FIFO is written in `aclk` and read in `pclk`.

Every crossing goes through one of the two FIFOs. (The old `wakeup_request` level synchronizer is gone with that port: PWAKEUP is a Requester output and enters on the `pclk` side, so it needs no crossing here.)

7.4.2 CDC FIFO Pointer Encoding

The wrapper derives `CDC_FIFO_DEPTH = (DEPTH < 4) ? 4 : DEPTH` and hands that to two `gaxi_fifo_async` instances (command and response). Gray pointers only close on a power-of-2 depth, so a FIFO depth of 6 cannot use Gray — `gaxi_fifo_async` carries an elaboration-time `$error` for exactly that case.

`USE_JOHNSON` selects the encoding:

USE_JOHNSON	Encoding	Pointer width	Depth constraint
`0` (default)	Gray	`$clog2(max(DEPTH, 4))+1`	floored `max(DEPTH, 4)` must be a power of 2: DEPTH in {2,

USE_JOHNSON	Encoding	Pointer width	Depth constraint
			3, 4, 8}
1	Johnson	`max(DEPTH, 4)` bits	any depth
-1	auto	per depth	none — Gray when the derived FIFO depth is a power of 2, Johnson otherwise

The default is Gray, not auto. With defaults, DEPTH 2, 3, 4 and 8 elaborate (2, 3 and 4 all derive a depth-4 FIFO) and **DEPTH 5, 6 and 7 fail the build**. That is intentional. Johnson costs `max(DEPTH, 4)`-bit pointers against Gray's `$clog2(max(DEPTH, 4))+1` (widths follow the floored FIFO depth, so the default `DEPTH = 2` pays 4 bits vs 3) — at depth 6, 6 bits against 4 — duplicated in both domains and again in every synchronizer stage. Nobody should pay that because a default quietly decided for them.

If you want a non-power-of-2 depth, say so: pass `USE_JOHNSON=1` for Johnson, or `USE_JOHNSON=-1` to restore per-depth auto-selection. The capability is intact; only the default changed, so that the choice is visible in the instantiation.

`USE_JOHNSON=0` with `DEPTH=6` is an elaboration error, deliberately. Asking for Gray at a non-power-of-2 depth is a real configuration mistake and should fail the build.

The pointer encoding is selectable, and defaults to Gray. `USE_JOHNSON` feeds a localparam that resolves against the DERIVED depth:

```
localparam int CDC_FIFO_DEPTH  = (DEPTH < 4) ? 4 : DEPTH;
localparam bit CDC_DEPTH_POW2  = ((CDC_FIFO_DEPTH & (CDC_FIFO_DEPTH -
1)) == 0);
localparam int CDC_USE_JOHNSON = (USE_JOHNSON >= 0) ? USE_JOHNSON
                                                    :
(CDC_DEPTH_POW2 ? 0 : 1);
```

With the default `USE_JOHNSON = 0`, `CDC_USE_JOHNSON` is 0 and the crossing is Gray — the default `DEPTH = 2` derives 4, a power of 2, so it elaborates. `DEPTH = 6` would not; pass `USE_JOHNSON = 1` for 6-bit Johnson pointers, or `-1` to let the derived depth decide. See the CDC FIFO pointer encoding section above for the full table.

7.4.3 Reset Synchronization

`presetn` and `aresetn` are independent reset domains and either may be asserted alone. **A one-sided reset is not safe. Quiesce the bus first.**

The local reset clears that domain's own pointer, but the crossed copy of the *remote* pointer is a live synchronizer (`glitch_free_n_dff_arn`, N=2) that keeps sampling the non-reset domain the moment reset deasserts — it does not hold at zero. The reset side therefore comes back with its own pointer at zero and the remote pointer at whatever the other side had reached, which is not an empty FIFO. It is a mismatched one.

Two concrete consequences, neither of which is a clean discard:

- **Consumed commands are re-presented.** Pulsing `aresetn` with an **already-consumed** command in the cmd FIFO rewinds the read pointer behind the write pointer, so the backend sees that command again after reset. It does not time out — it re-executes. An *unread* command is not at risk: its read pointer is already behind the write pointer, so resetting it to 0 rewinds nothing and the entry is delivered exactly once.
- **The response FIFO can fabricate responses.** The same rewind on the response path presents entries the APB side never queued, so the APB master can complete a transfer that the backend did not answer.

Pointers being absolute positions rather than toggle parity is what makes the *steady-state* crossing reliable; it does not make a one-sided reset safe.

Neither reset is internally synchronized to the other domain's clock; each is expected to be already synchronized (or asynchronously asserted and synchronously deasserted) in its own domain by the integrator.

7.5 Timing Characteristics

7.5.1 Latency

- Crossing latency is the async-FIFO write-to-read pointer synchronization: on the order of 2-3 destination-clock cycles per direction, plus the source-clock cycle that performs the write
- A full APB transfer therefore costs the command crossing plus the response crossing before PREADY can assert
- Additional latency if the FIFOs are full (backpressure) or the backend is slow

7.5.2 Timing Considerations

Timing diagram pending. The signals and sequence this scenario exercises:

- pclk
- aclk (different frequency)
- APB transaction signals
- FIFO write/read pointers
- Backend transaction signals
- CDC latency

7.6 Usage Examples

```
apb5_slave_cdc #(
    .ADDR_WIDTH     (32),
    .DATA_WIDTH     (32),
    .AUSER_WIDTH    (4),
    .WUSER_WIDTH    (4),
    .RUSER_WIDTH    (4),
    .BUSER_WIDTH    (4),
    .DEPTH          (2),
    .ENABLE_PARITY  (0)
) u_apb5_slave_cdc (
    // APB clock domain
    .pclk           (apb_clk),
    .presetn        (apb_rst_n),

    // Backend clock domain
    .aclk           (backend_clk),
    .aresetn        (backend_rst_n),

    // APB5 slave interface (pclk domain)
    .s_apb_PSEL     (s_apb_psel),
    .s_apb_PENABLE  (s_apb_penable),
    // ... other APB signals

    // Backend interface (aclk domain)
    .cmd_valid      (backend_cmd_valid),
    .cmd_ready      (backend_cmd_ready)
    // ... other backend signals
);
```

7.7 Design Notes

7.7.1 FIFO Depth Sizing

- `DEPTH` sizes the `apb5_slave` skid buffers; the CDC FIFOs are sized separately as `max(DEPTH, 4)`, so the crossing itself is never shallower than 4 entries even at the default `DEPTH=2`
- Deeper FIFOs buy tolerance for a slow backend or a bursty APB master; they do not reduce the per-transfer crossing latency
- Because APB is single-outstanding, depth beyond a handful of entries has little effect on throughput for this module

---

7.8 Related Modules

- **APB5 Slave** - Base slave without CDC
- **APB5 Slave CDC CG** - CDC with clock gating
- **GAXI Async FIFO** - The async FIFO used for both crossings
- **Clock Domain Crossing** - CDC design patterns and reset behavior

---

7.9 Testing

`val/amba/test_apb5_slave_cdc.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_apb5_slave_cdc.py -v
```

7.10 References

- Arm. *AMBA APB Protocol Specification*, ARM IHI 0024E, Feb. 2023 (APB5 first published in Issue D, Apr. 2021). developer.arm.com/documentation/ihi0024/e — ch. 3 “Transfers”, ch. 4 “Operating States”, §3.7 “Wake-up signaling” and §3.8 “User signaling”: the protocol of the wrapped APB5 block. Citation only.
- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 — the Gray-pointer async FIFO (`gaxi_fifo_async`) that carries the command and response across the two clocks.

- Ginosar, R. “Metastability and Synchronizers: A Tutorial.” *IEEE Design & Test of Computers*, 28(5), Sept./Oct. 2011. DOI: 10.1109/MDT.2011.113 — synchronizer depth versus MTBF for the pointer crossings.
- Deschamps, J.-P., Sutter, G.D., Cantó, E. *Guide to FPGA Implementation of Arithmetic Functions*, Springer (LNEE 149), 2012, ch. 5 “Electronic Aspects of Digital Design”, §5.3.5.3 “Asynchronous FIFO” — textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/cdc/references/AsyncFifoGrayPointers__Cummings2002.pdf`, `rtl/cdc/references/MetastabilitySynchronizers__Ginosar2011.pdf`)

7.11 Navigation

- **← Back to APB5 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

8 APB5 Slave CDC (Clock-Gated)

Module: `apb5_slave_cdc_cg.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

8.1 Overview

The APB5 Slave CDC + Clock-Gated module combines clock domain crossing with clock gating for maximum power efficiency. It wraps `apb5_slave_cdc` with clock gating control to cut power during idle periods while keeping operation safe across asynchronous clock boundaries.

8.1.1 Key Features

- Full APB5 protocol support with all extensions
- Asynchronous clock domain crossing (APB to backend)
- Clock gating for power reduction during idle
- All APB5 user signals (PAUSER, PWUSER, PRUSER, PBUSER)
- Wakes the `pclk` clock gate on the Requester's PWAKEUP (IHI 0024E §3.7.2)
- Optional IHI 0024E ch. 5 interface parity (`*CHK` signals, odd parity)
- Automatic wake-up on transaction activity

8.2 Parameters

Parameter	Type	Default	Description
`ADDR_WIDTH`	int	32	APB address bus width
`DATA_WIDTH`	int	32	APB data bus width
`PROT_WIDTH`	int	3	Protection signal width
`AUSER_WIDTH`	int	4	Address user signal width
`WUSER_WIDTH`	int	4	Write user signal width
`RUSER_WIDTH`	int	4	Read user signal width
`BUSER_WIDTH`	int	4	Response user signal width
`STRB_WIDTH`	int	DATA_WIDTH/8	Write strobe width (calculated)
`DEPTH`	int	2	Skid-buffer depth of the

Parameter	Type	Default	Description
			wrapped slave; CDC FIFOs use `max(DEPTH, 4)`
`USE_JOHNSON`	int	0 (Gray)	Forwarded to `apb5_slave_cdc`: `0` Gray, `1` Johnson, `-1` auto. Newly reachable from this level — it used to be unreachable, so a CG instance could not select the encoding at all.
`ENABLE_PARITY`	bit	0	Enable parity generation and checking
`CG_IDLE_COUNT_WIDTH`	int	4	Width of idle counter (max idle = 2^N-1 cycles)
`USE_2_PHASE_CDC`	bit	1	Deprecated and ignored — retained for source compatibility

As with apb5_slave_cdc, there is no `SYNC_STAGES` parameter: pointer synchronization is fixed at 2 flops inside the async FIFOs.

8.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`DW`	`DATA_WIDTH`
`AW`	`ADDR_WIDTH`
`SW`	`STRB_WIDTH`
`PW`	`PROT_WIDTH`
`AUW`	`AUSER_WIDTH`
`WUW`	`WUSER_WIDTH`
`RUW`	`RUSER_WIDTH`
`BUW`	`BUSER_WIDTH`
`ICW`	`CG_IDLE_COUNT_WIDTH`

Derived parameter	Default expression
CPW	AW + DW + SW + PW + AUW + WUW + 1
RPW	DW + RUW + BUW + 1

8.3 Ports

8.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB bus clock
presetn	1	Input	APB reset (active low)
aclk	1	Input	User/backend clock
aresetn	1	Input	User reset (active low)

8.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating

8.3.3 APB5 Slave Interface

Same as apb5_slave - operates in pclk domain.

8.3.4 Backend Interface

Same command/response interface as apb5_slave_cdc - operates in aclk domain.

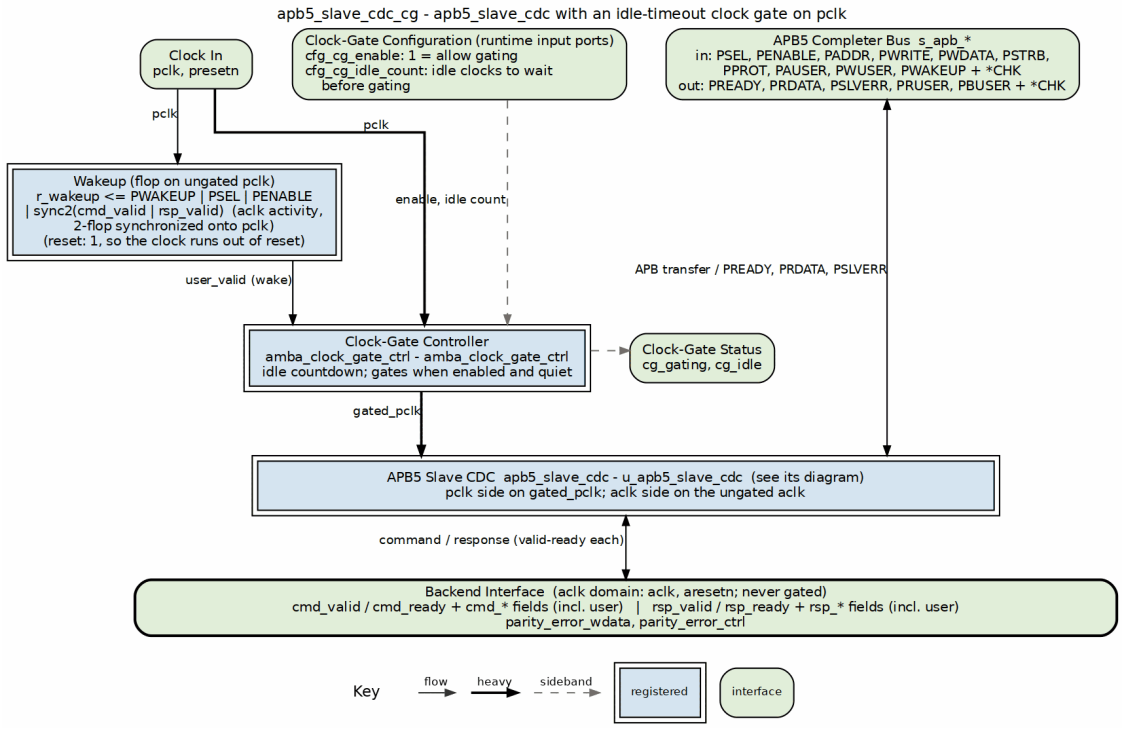
8.3.5 Status Outputs

Port	Width	Direction	Description
cg_gating	1	Output	Indicates clock is currently gated
cg_idle	1	Output	Activity terms quiet

Port	Width	Direction	Description
			(registered ~wakeup).
parity_error_wdata	1	Output	PWDATACHK, PSTRBCHK or PWUSERCHK mismatch (gated-pclk domain)
parity_error_ctrl	1	Output	PADDRCHK, PCTRLCHK, PSELCHK, PENABLECHK or PAUSERCHK mismatch (gated-pclk domain)

8.4 Functional Description

8.4.1 Architecture



apb5_slave_cdc_cg block diagram

The whole `apb5_slave_cdc` is one box (see its diagram). One clock gate, on `pclk` only: its wakeup flop ORs PWAKEUP, PSEL and PENABLE with the `aclk`-side activity (`cmd_valid | rsp_valid`), which is brought onto `pclk` through a two-flop synchronizer first. The gated clock (heavy) runs the pclk side; the aclk side is never gated.

8.4.2 Wake-up Logic

The wakeup terms are in the block diagram's Wakeup box. The `aclk`-domain activity terms are ORed together and passed through a two-flop synchronizer before being combined with the `pclk`-domain terms. Without that synchronizer, these signals would cross domains unsynchronized.

8.5 Timing Characteristics

Timing diagram pending. The signals and sequence this scenario exercises:

- pclk (ungated)
- gated_pclk
- aclk (different frequency)
- s_apb_PSEL (wake trigger)
- cg_gating indicator
- Transaction flow across CDC with gating
- Wake-up latency from PSEL to clock active

8.6 Usage Examples

```
apb5_slave_cdc_cg #(
    .ADDR_WIDTH         (32),
    .DATA_WIDTH         (32),
    .AUSER_WIDTH        (4),
    .WUSER_WIDTH        (4),
    .RUSER_WIDTH        (4),
    .BUSER_WIDTH        (4),
    .DEPTH              (2),
    .ENABLE_PARITY      (0),
    .CG_IDLE_COUNT_WIDTH(4)
) u_apb5_slave_cdc_cg (
    // APB clock domain
    .pclk               (apb_clk),
    .presetn            (apb_rst_n),

    // User clock domain
    .aclk               (user_clk),
    .aresetn            (user_rst_n),

    // Clock gating
```

```
    .cfg_cg_enable      (1'b1),
    .cfg_cg_idle_count  (4'd8),
    .cg_gating   (slave_clk_gated),

    // APB5 slave interface (pclk domain)
    .s_apb_PSEL         (s_apb_psel),
    .s_apb_PENABLE      (s_apb_penable),
    .s_apb_PREADY       (s_apb_pready),
    // ... other APB signals

    // Backend interface (aclk domain)
    .cmd_valid          (backend_cmd_valid),
    .cmd_ready          (backend_cmd_ready),
    // ... other command signals

    .rsp_valid          (backend_rsp_valid),
    .rsp_ready          (backend_rsp_ready)
    // ... other response signals
);
```

8.7 Design Notes

8.7.1 Power and Latency Trade-offs

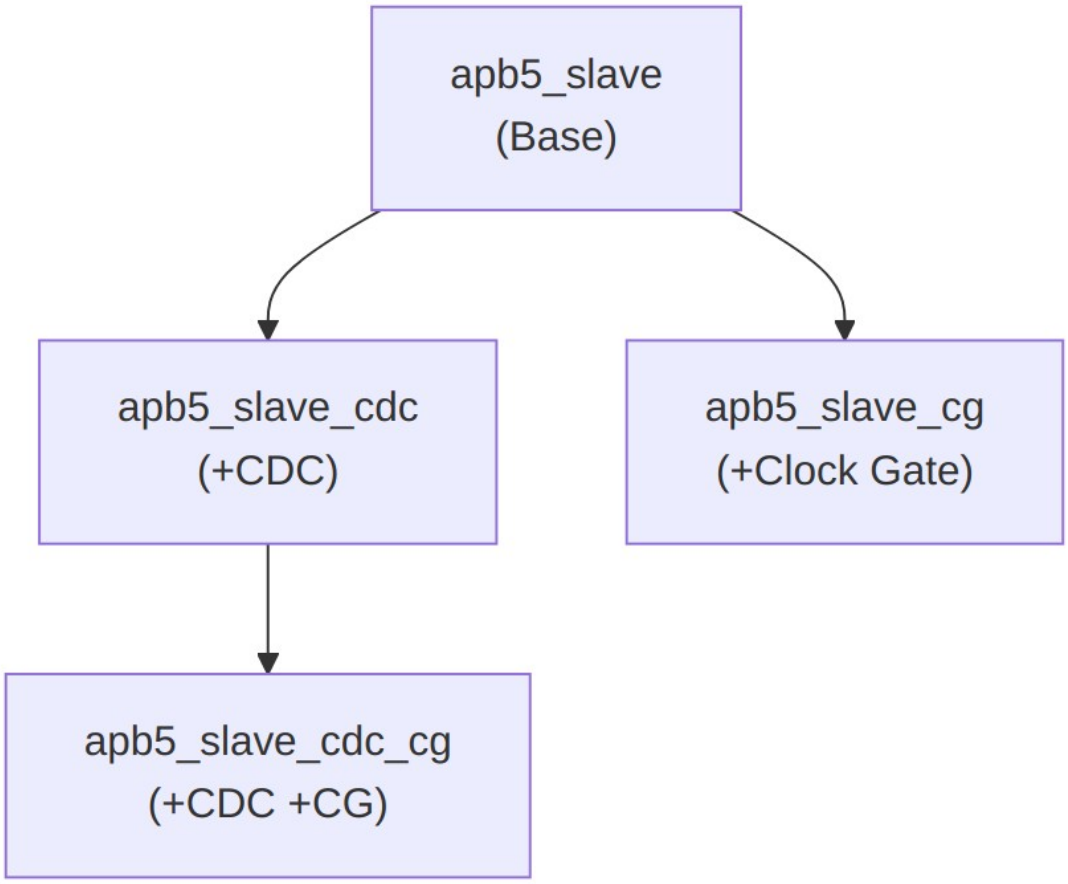
Wake-up is **registered, not combinational**. Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream, APB) or twice (APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. A pclk-domain trigger (PWAKEUP, PSEL or PENABLE) passes through this wrapper's `r_wakeup` and then through the flop inside `amba_clock_gate_ctrl`, so APB5 is a two-stage family and the first usable gated `pclk` rising edge arrives **3 ungated `pclk` cycles** after the trigger. An aclk-domain trigger (`cmd_valid`, `rsp_valid`) first crosses a 2-flop synchronizer into `pclk`, adding about two more `pclk` cycles.

Configuration	Power Savings	Wake-up Latency from PSEL
CG disabled	None	0 cycles (clock always running)
CG idle=4	Moderate	2 register stages; first usable edge 3 ungated pclk cycles
CG idle=16 [1]	Good	2 register stages; first usable edge 3 ungated pclk cycles

[1] `idle=16` requires `CG_IDLE_COUNT_WIDTH >= 5`. At the default of 4 the counter holds 0-15, so writing 16 truncates to 0 and gates after the first idle cycle – the opposite of the intent. Raise the parameter for this row.

The wake-up latency does not depend on `cfg_cg_idle_count`; the idle count only controls how long the block waits before gating. Latency is absorbed as APB wait states, since the slave holds PREADY low until it is running again.

Deliberately keeping the wake-up path registered avoids driving a clock-gate enable from a combinational function of a bus input. The gating element itself is an ICG cell instantiated by `clock_gate_ctrl`, which is glitch-free by construction. ICG cells are an ASIC library primitive; FPGA targets should use a clock-enable approach instead.



Combined Feature Hierarchy

8.7.2 Reset Considerations

- APB domain reset (`presetn`) controls the APB interface, the wake-up synchronizer and the clock gate
- User domain reset (`aresetn`) controls the backend side of the CDC FIFOs
- Each reset must already be synchronized (or asynchronously asserted and synchronously deasserted) in its own domain by the integrator; this module does not synchronize either reset into the other domain

- A one-sided reset is NOT safe: the crossed pointer copy is a live synchronizer, so the reset side returns with its own pointer at zero against a remote pointer that kept advancing. Quiesce the bus first — see apb5_slave_cdc for the mechanism and for the in-flight transaction caveat

8.8 Related Modules

- **APB5 Slave** - Base slave module
- **APB5 Slave CDC** - CDC variant without clock gating
- **APB5 Slave CG** - Clock gating without CDC

8.9 Testing

`val/amba/test_apb5_slave_cdc_cg.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_apb5_slave_cdc_cg.py -v
```

8.10 References

- Arm. *AMBA APB Protocol Specification*, ARM IHI 0024E, Feb. 2023 (APB5 first published in Issue D, Apr. 2021). developer.arm.com/documentation/ihi0024/e — ch. 3 “Transfers”, ch. 4 “Operating States”, §3.7 “Wake-up signaling” and §3.8 “User signaling”: the protocol of the wrapped APB5 block. Citation only.
- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 — the Gray-pointer async FIFO (`gaxi_fifo_async`) that carries the command and response across the two clocks.
- Ginosar, R. “Metastability and Synchronizers: A Tutorial.” *IEEE Design & Test of Computers*, 28(5), Sept./Oct. 2011. DOI: 10.1109/MDT.2011.113 — synchronizer depth versus MTBF for the pointer crossings.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 — the fixed-timeout policy `amba_clock_gate_ctrl` implements: gate after `cfg_cg_idle_count` idle cycles, wake on activity.

- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. CAD*, 15(6), June 1996. DOI: 10.1109/43.503933 — the latch-based gated clock (`icg`) at the bottom of the gating chain.
- Deschamps, J.-P., Sutter, G.D., Cantó, E. *Guide to FPGA Implementation of Arithmetic Functions*, Springer (LNEE 149), 2012, ch. 5 “Electronic Aspects of Digital Design”, §5.3.5.3 “Asynchronous FIFO” — textbook treatment (secondary source).
- Roy, S. *Advanced Digital System Design*, Springer, ch. 15 “Low-Power Digital System Design”, §15.5.5 “Logic Level Power Down” — textbook treatment of clock gating (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/cdc/references/AsyncFifoGrayPointers__Cummings2002.pdf`, `rtl/cdc/references/MetastabilitySynchronizers__Ginosar2011.pdf`, `rtl/common/references/DynamicPowerManagementTimeout__Benini2000.pdf`, `rtl/common/references/LatchBasedClockGating__Benini1996.pdf`)

8.11 Navigation

- **← Back to APB5 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

9 APB5 Slave (Clock-Gated)

Module: `apb5_slave_cg.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

9.1 Overview

Clock-gated variant of the APB5 Slave module. Wraps the base `apb5_slave` with clock gating control logic to reduce dynamic power consumption when no transactions are active.

9.1.1 Key Features

- All APB5 Slave features (see apb5_slave)
- Automatic clock gating during idle periods
- Registered wake-up on the Requester's PWAKEUP (ahead of PSEL) or on PSEL, through a glitch-free ICG cell
- Configurable idle threshold
- Gating status output

9.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address/ request user signal width
WUSER_WIDTH	int	4	Write data user signal width
RUSER_WIDTH	int	4	Read data user signal width
BUSER_WIDTH	int	4	Response user

Parameter	Type	Default	Description
			signal width
DEPTH	int	2	FIFO depth
ENABLE_PARITY	bit	0	Enable APB5 parity generation and checking
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter

All other parameters are inherited from apb5_slave.

9.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
STRB_WIDTH	DATA_WIDTH / 8
DW	DATA_WIDTH
AW	ADDR_WIDTH
SW	STRB_WIDTH
PW	PROT_WIDTH
AUW	AUSER_WIDTH
WUW	WUSER_WIDTH
RUW	RUSER_WIDTH
BUW	BUSER_WIDTH
ICW	CG_IDLE_COUNT_WIDTH
CPW	AW + DW + SW + PW + AUW + WUW + 1
RPW	DW + RUW + BUW + 1

9.3 Ports

9.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low reset

9.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating

9.3.3 Clock Gating Status

Port	Width	Direction	Description
cg_gating	1	Output	High while the internal clock is gated off
cg_idle	1	Output	Activity terms quiet (registered ~wakeup).

There is no cumulative gated-cycle counter port on this module. If a gated-cycle total is needed, count `cg_gating` in the integrating logic on the ungated `pclk`.

All ports of apb5_slave – including the parity signals, `s_apb_PWAKEUP`, `parity_error_wdata` and `parity_error_ctrl` – are present unchanged and pass straight through to the wrapped core.

9.3.4 APB5 Slave Interface

Port	Width	Direction	Description
s_apb_PSEL	1	Input	APB select signal

Port	Width	Direction	Description
s_apb_PENABLE	1	Input	APB enable signal
s_apb_PREADY	1	Output	Slave ready response
s_apb_PADDR	AW	Input	APB address
s_apb_PWRITE	1	Input	Write/read indicator
s_apb_PWDATA	DW	Input	Write data
s_apb_PSTRB	SW	Input	Write byte strobes
s_apb_PPROT	PW	Input	Protection attributes
s_apb_PAUSER	AUW	Input	User-defined request attributes
s_apb_PWUSER	WUW	Input	User-defined write data attributes
s_apb_PRDATA	DW	Output	Read data to master
s_apb_PSLVERR	1	Output	Slave error response
s_apb_PWAKEUP	1	Input	Requester wake-up (IHI 0024E §3.7.2): a clock-gate wake term in this wrapper
s_apb_PRUSER	RUW	Output	User-defined read data attributes
s_apb_PBUSER	BUW	Output	User-defined response attributes

9.3.5 Parity Signals (Optional)

Port	Width	Direction	Description
s_apb_PADDRCHK	ceil(ADDR_WIDTH/8)	Input	Odd parity over PADDR, one bit per 8 bits
s_apb_PCTRLCHK	1	Input	Odd parity over PPROT and PWRITE (PNSE not carried)
s_apb_PSELCHK	1	Input	Odd parity over PSEL (the spec's PSELxCHK)
s_apb_PENABLECHK	1	Input	Odd parity over PENABLE
s_apb_PWDATACHK	DATA_WIDTH/8	Input	Odd parity over PWDATA, one bit per byte lane
s_apb_PSTRBCHK	1	Input	Odd parity over all of PSTRB
s_apb_PAUSERCHK	ceil(AUSER_WIDTH/8)	Input	Odd parity over PAUSER, one bit per 8 bits
s_apb_PWUSERCHK	ceil(WUSER_WIDTH/8)	Input	Odd parity over PWUSER, one bit per 8 bits
s_apb_PRDATACHK	DATA_WIDTH/8	Output	Odd parity over PRDATA, one bit per byte lane
s_apb_PREADYCHK	1	Output	Odd parity over PREADY
s_apb_PSLVERRCHK	1	Output	Odd parity over PSLVERR
s_apb_PWAKEUPCHK	1	Input	Odd parity over PWAKEUP
s_apb_PRUSERCHK	ceil(RUSER_WIDTH/8)	Output	Odd parity over PRUSER, one bit per 8 bits
s_apb_PBUSERCHK	ceil(BUSER_WIDTH/8)	Output	Odd parity over PBUSER, one bit per 8 bits

9.3.6 Command Interface (to backend)

Port	Width	Direction	Description
cmd_valid	1	Output	Command valid to backend
cmd_ready	1	Input	Backend ready
cmd_pwrite	1	Output	Command write/read
cmd_paddr	AW	Output	Command address
cmd_pwdata	DW	Output	Command write data
cmd_pstrb	SW	Output	Command write strobes
cmd_pprot	PW	Output	Command protection
cmd_pauser	AUW	Output	Command address user
cmd_pwuser	WUW	Output	Command write user

9.3.7 Response Interface (from backend)

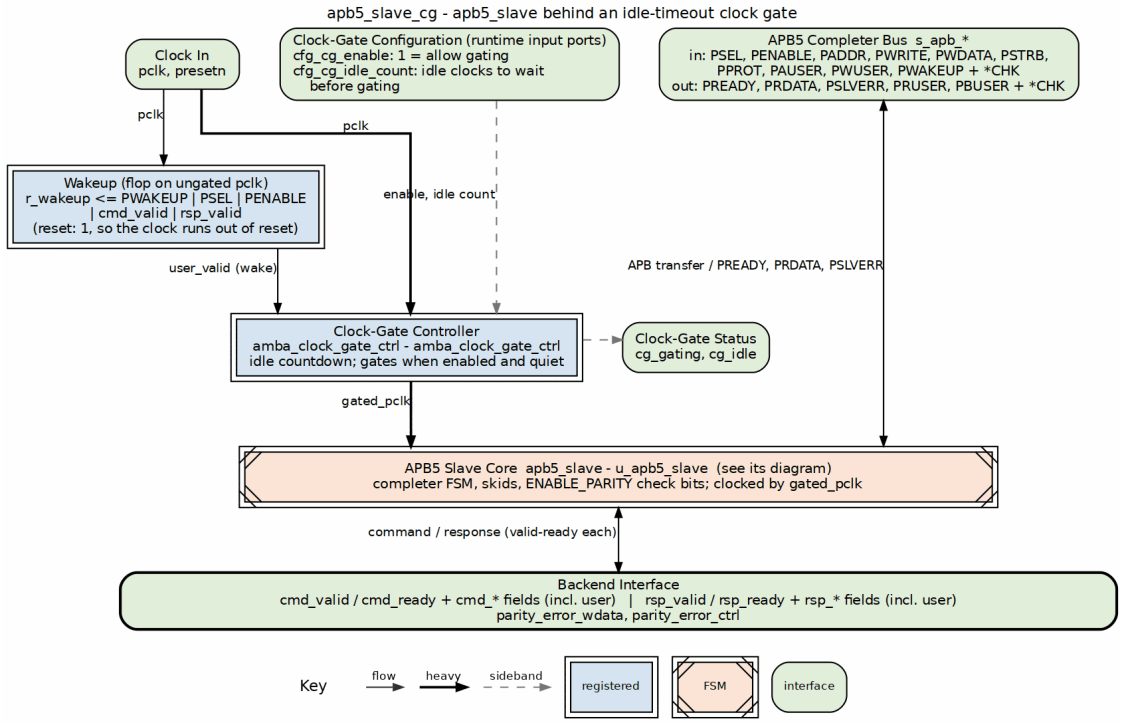
Port	Width	Direction	Description
rsp_valid	1	Input	Response valid from backend
rsp_ready	1	Output	Slave ready for response
rsp_prdata	DW	Input	Response read data
rsp_pslverr	1	Input	Response error status
rsp_pruser	RUW	Input	Response read user
rsp_pbuser	BUW	Input	Response user

9.3.8 Status Outputs

Port	Width	Direction	Description
parity_error_wdata	1	Output	PWDATACHK, PSTRBCHK or PWUSERCHK mismatch (tied to 0 when ENABLE_PARITY=0)
parity_error_ctrl	1	Output	PADDRCHK, PCTRLCHK, PSELCHK, PENABLECHK, PAUSERCHK or PWAKEUPCHK mismatch (tied to 0 when ENABLE_PARITY=0)

9.4 Functional Description

9.4.1 Block diagram



apb5_slave_cg block diagram

The wrapped `apb5_slave` is one box (see its diagram). The variant adds the clock path (heavy): a wakeup flop on the ungated `pclk` watches PWAKEUP, PSEL, PENABLE, `cmd_valid` and

rsp_valid, and amba_clock_gate_ctrl stops the core's clock after cfg_cg_idle_count quiet clocks when cfg_cg_enable is set.

9.4.2 Clock Gating Behavior

9.4.2.1 Wake-up Trigger

The wrapper keeps the clock running whenever any of the following is high:

```
s_apb_PWAKEUP || s_apb_PSEL || s_apb_PENABLE || cmd_valid || rsp_valid
```

s_apb_PWAKEUP is the Requester's wake-up (IHI 0024E §3.7.2), which the spec intends to be routed to a clock controller. A Requester that raises it a cycle or more ahead of PSEL (as §3.7.2 recommends, and as apb5_master does) gets the clock restarted before the setup phase arrives. The term is this side's pending work and the Requester's wake request only – never a peer READY.

That term is registered into r_wakeup on the ungated pclk, and amba_clock_gate_ctrl registers it once more before it reaches the gating condition. Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. APB5 is a **two-stage** family, so the first gated-clock rising edge available to the wrapped apb5_slave arrives **3 ungated pclk cycles** after activity asserts. Because the slave signals not-ready by holding PREADY low, those cycles simply appear to the master as APB wait states.

The gating element is an ICG (integrated clock gating) cell instantiated by clock_gate_ctrl, so enable transitions are glitch-free. ICG cells are an ASIC library primitive; on FPGA targets use a clock-enable approach instead.

Gating engages cfg_cg_idle_count + 1 ungated pclk cycles after the internal wakeup deasserts, which is cfg_cg_idle_count + 3 cycles after the last bus activity, because APB5 adds two register stages ahead of the ICG enable.

9.5 Timing Characteristics

Timing diagram pending. The signals and sequence this scenario exercises:

- pclk
- gated_pclk
- s_apb_PSEL
- cg_gating
- Wake-up latency (two register stages; first usable gated edge 3 ungated pclk cycles after activity)

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

9.6 Usage Examples

```
apb5_slave_cg #(
    .ADDR_WIDTH         (32),
    .DATA_WIDTH         (32),
    .CG_IDLE_COUNT_WIDTH(4)
) u_apb5_slave_cg (
    .pclk               (apb_clk),
    .presetn            (apb_rst_n),

    // Clock gating
    .cfg_cg_enable      (1'b1),
    .cfg_cg_idle_count  (4'd4),
    .cg_gating   (slave_clk_gated),

    // APB5 interface (same as apb5_slave)
    // ...
);
```

9.7 Design Notes

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into `user_valid` pins this block permanently awake and defeats gating entirely – silently, because function is unaffected. Ten wrappers in this repository shipped that way and nothing failed until someone measured. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating`. Canonical rule: `vault/handbook/design/clock-gating-activity-terms.md`.

`cfg_cg_enable` is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only; the datapath and any monitor enables are forwarded untouched. With it low the clock free-runs and this module behaves exactly like its base.

Gating latency. The clock stops `cfg_cg_idle_count` + 2 cycles after the last bus activity – the idle counter, plus one for the `r_wakeup` flop. Size the idle count against your traffic's inter-burst gap: too small and the block wakes constantly, too large and it never gates.

Cost. Five flops: `r_wakeup` plus `r_idle_counter` at `IDLE_CNTR_WIDTH`, scaling as 1 + `CG_IDLE_COUNT_WIDTH`. The ICG itself is a latch or BUFGCE, not fabric flops.

9.8 Related Modules

- **APB5 Slave** - Base module documentation
- **APB5 Master CG** - Clock-gated master

9.9 Testing

`val/amba/test_apb5_slave_cg.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_apb5_slave_cg.py -v
```

9.10 References

- Arm. *AMBA APB Protocol Specification*, ARM IHI 0024E, Feb. 2023 (APB5 first published in Issue D, Apr. 2021). developer.arm.com/documentation/ihi0024/e – ch. 3 "Transfers", ch. 4 "Operating States", §3.7 "Wake-up signaling" and §3.8 "User signaling": the protocol of the wrapped APB5 block. Citation only.
- Benini, L., Bogliolo, A., De Micheli, G. "A Survey of Design Techniques for System-Level Dynamic Power Management." *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – the fixed-timeout policy `amba_clock_gate_ctrl` implements: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Benini, L., De Micheli, G. "Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines." *IEEE Trans. CAD*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based gated clock (`icg`) at the bottom of the gating chain.
- Roy, S. *Advanced Digital System Design*, Springer, ch. 15 "Low-Power Digital System Design", §15.5.5 "Logic Level Power Down" – textbook treatment of clock gating (secondary source).

- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/common/references/DynamicPowerManagementTimeout__Benini2000.pdf`, `rtl/common/references/LatchBasedClockGating__Benini1996.pdf`)

9.11 Navigation

- **← Back to APB5 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

10 APB5 Slave Stub

Module: `apb5_slave_stub.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

10.1 Overview

The APB5 Slave Stub provides a simplified packed-data interface for responding to APB5 transactions. It implements a complete APB5 slave with packet-based command/response interfaces, making it ideal for testbenches and simple backend implementations.

10.1.1 Key Features

- Packed command/response packet interface
- Complete APB5 slave FSM implementation
- All APB5 extensions (PAUSER, PWUSER, PRUSER, PBUSER)
- PWAKEUP input from the Requester (IHI 0024E §3.7.2), checked by PWAKEUPCHK
- Optional parity support with error detection
- Skid buffers for command and response paths

10.2 Parameters

Parameter	Type	Default	Description
DEPTH	int	4	Skid-buffer depth in entries; 2..8 inclusive (any integer)
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address user signal width
WUSER_WIDTH	int	4	Write user signal width
RUSER_WIDTH	int	4	Read user signal width
BUSER_WIDTH	int	4	Response user signal width

Parameter	Type	Default	Description
ENABLE_PARITY	bit	0	Enable parity signals
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width
CMD_PACKET_WIDTH	int	(calculated)	Total command packet width
RESP_PACKET_WIDTH	int	(calculated)	Total response packet width

`CMD_PACKET_WIDTH` and `RESP_PACKET_WIDTH` are computed from the width parameters and must not be overridden:

```
CMD_PACKET_WIDTH  = ADDR_WIDTH + DATA_WIDTH + STRB_WIDTH + PROT_WIDTH
                  + AUSER_WIDTH + WUSER_WIDTH + 1
                    // the single bit is pwrite

RESP_PACKET_WIDTH = DATA_WIDTH + RUSER_WIDTH + BUSER_WIDTH + 1
                    // the single bit is pslverr
```

With all defaults this gives `CMD_PACKET_WIDTH = 80` and `RESP_PACKET_WIDTH = 41`.

Unlike apb5_master_stub, the slave stub packets carry no `first`/`last` markers, so the two stubs' packet widths are not interchangeable.

10.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
DW	DATA_WIDTH
AW	ADDR_WIDTH
SW	STRB_WIDTH
PW	PROT_WIDTH
AUW	AUSER_WIDTH
WUW	WUSER_WIDTH
CPW	CMD_PACKET_WIDTH
RPW	RESP_PACKET_WIDTH

10.3 Ports

10.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB reset (active low)

10.3.2 APB5 Slave Interface

Port	Width	Direction	Description
s_apb_PSEL	1	Input	APB select signal
s_apb_PENABLE	1	Input	APB enable signal
s_apb_PADDR	ADDR_WIDTH	Input	APB address
s_apb_PWRITE	1	Input	Write/read indicator
s_apb_PWDATA	DATA_WIDTH	Input	Write data
s_apb_PSTRB	STRB_WIDTH	Input	Write byte strobes
s_apb_PPROT	PROT_WIDTH	Input	Protection attributes
s_apb_PAUSER	AUSER_WIDTH	Input	User request attributes
s_apb_PWUSER	WUSER_WIDTH	Input	User write attributes
s_apb_PRDATA	DATA_WIDTH	Output	Read data to master
s_apb_PSLVERR	1	Output	Slave error response
s_apb_PREADY	1	Output	Slave ready

Port	Width	Direction	Description
s_apb_PWAKEUP	1	Input	Requester wake-up (IHI 0024E §3.7.2); not waited on before PREADY
s_apb_PRUSER	RUSER_WIDTH	Output	User read attributes
s_apb_PBUSER	BUSER_WIDTH	Output	User response attributes

10.3.3 Parity Signals (Optional)

Present unconditionally; meaningful only when `ENABLE_PARITY=1`. Semantics are identical to apb5_slave.

Port	Width	Direction	Description
s_apb_PADDRCHK	ceil(ADDR_WIDTH/8)	Input	Odd parity over PADDR, one bit per 8 bits
s_apb_PCTRLCHK	1	Input	Odd parity over PPROT and PWRITE (PNSE not carried)
s_apb_PSELCHK	1	Input	Odd parity over PSEL (the spec's PSELxCHK)
s_apb_PENABLECHK	1	Input	Odd parity over PENABLE
s_apb_PWDATACHK	DATA_WIDTH/8	Input	Odd parity over PWDATA, one bit per byte lane
s_apb_PSTRBCHK	1	Input	Odd parity over all of PSTRB
s_apb_PAUSERCHK	ceil(AUSER_WIDTH/8)	Input	Odd parity over PAUSER, one bit per 8 bits
s_apb_PWUSERCHK	ceil(WUSER_WIDTH/8)	Input	Odd parity over PWUSER, one bit per 8 bits
s_apb_PRDATACHK	DATA_WIDTH/8	Output	Odd parity over PRDATA, one bit per byte lane
s_apb_PREADYCHK	1	Output	Odd parity over PREADY

Port	Width	Direction	Description
s_apb_PSLVERRCHK	1	Output	Odd parity over PSLVERR
s_apb_PWAKEUPCHK	1	Input	Odd parity over PWAKEUP
s_apb_PRUSERCHK	ceil(RUSER_WIDTH/8)	Output	Odd parity over PRUSER, one bit per 8 bits
s_apb_PBUSERCHK	ceil(BUSER_WIDTH/8)	Output	Odd parity over PBUSER, one bit per 8 bits

10.3.4 Command Packet Interface

Port	Width	Direction	Description
cmd_valid	1	Output	Command packet valid
cmd_ready	1	Input	Backend ready for command
cmd_data	CMD_PACKET_WIDTH	Output	Packed command data

10.3.5 Response Packet Interface

Port	Width	Direction	Description
rsp_valid	1	Input	Response packet valid
rsp_ready	1	Output	Ready for response
rsp_data	RESP_PACKET_WIDTH	Input	Packed response data

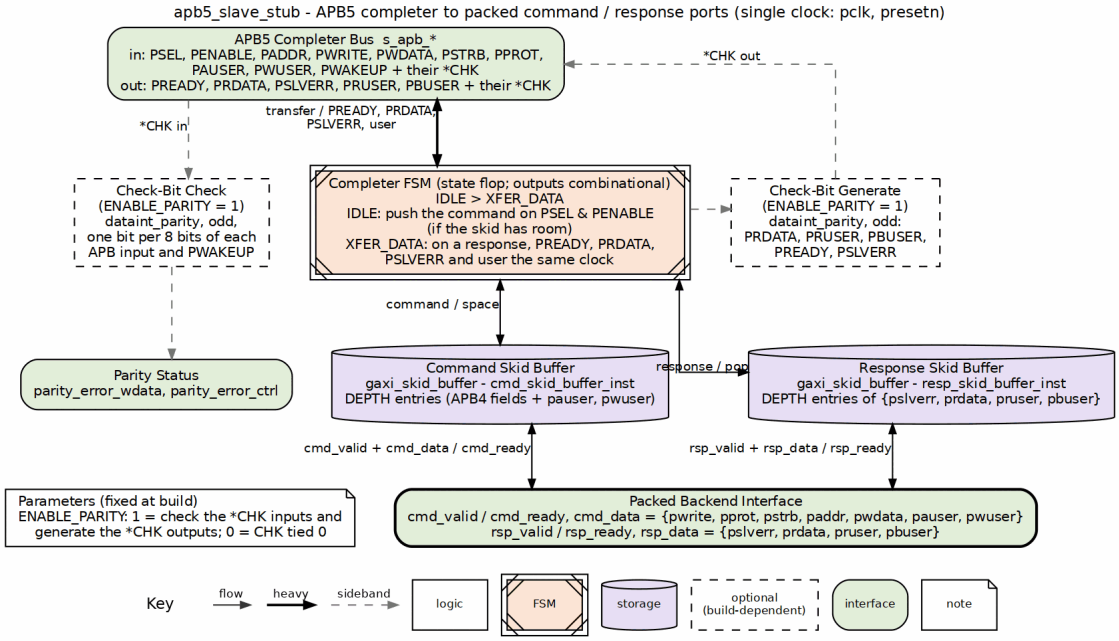
10.3.6 Control and Status

Port	Width	Direction	Description
parity_error_wdata	1	Output	PWDATACHK, PSTRBCHK or PWUSERCHK mismatch
parity_error_ctrl	1	Output	PADDRCHK, PCTRLCHK, PSELCHK, PENABLECHK,

Port	Width	Direction	Description
			PAUSERCHK or PWAKEUPCHK mismatch

10.4 Functional Description

10.4.1 Block diagram



apb5_slave_stub block diagram

A standalone completer, like `apb4_slave_stub`, with the user fields and the APB5 check bits added. A two-state FSM (IDLE > XFER_DATA) pushes the command into the command skid and, on a response, drives PREADY, PRDATA, PSLVERR and the user fields combinationally the same clock. With `ENABLE_PARITY = 1` the input *CHK are checked and the output* CHK generated.

10.4.2 Packet Formats

10.4.2.1 Command Packet Structure

`cmd_data = {pwrite, pprot, pstrb, paddr, pwdata, pauser, pwuser}`

Field	Width	Description
pwrite	1	Write/read indicator
pprot	PROT_WIDTH	Protection attributes
pstrb	STRB_WIDTH	Write byte strobes

Field	Width	Description
paddr	ADDR_WIDTH	Transaction address
pwdata	DATA_WIDTH	Write data
pauser	AUSER_WIDTH	Address user signal
pwuser	WUSER_WIDTH	Write user signal

10.4.2.2 Response Packet Structure

```
rsp_data = {pslverr, prdata, pruser, pbuser}
```

Field	Width	Description
pslverr	1	Error response
prdata	DATA_WIDTH	Read data
pruser	RUSER_WIDTH	Read user signal
pbuser	BUSER_WIDTH	Response user signal

```
stateDiagram-v2
    [*] --> IDLE

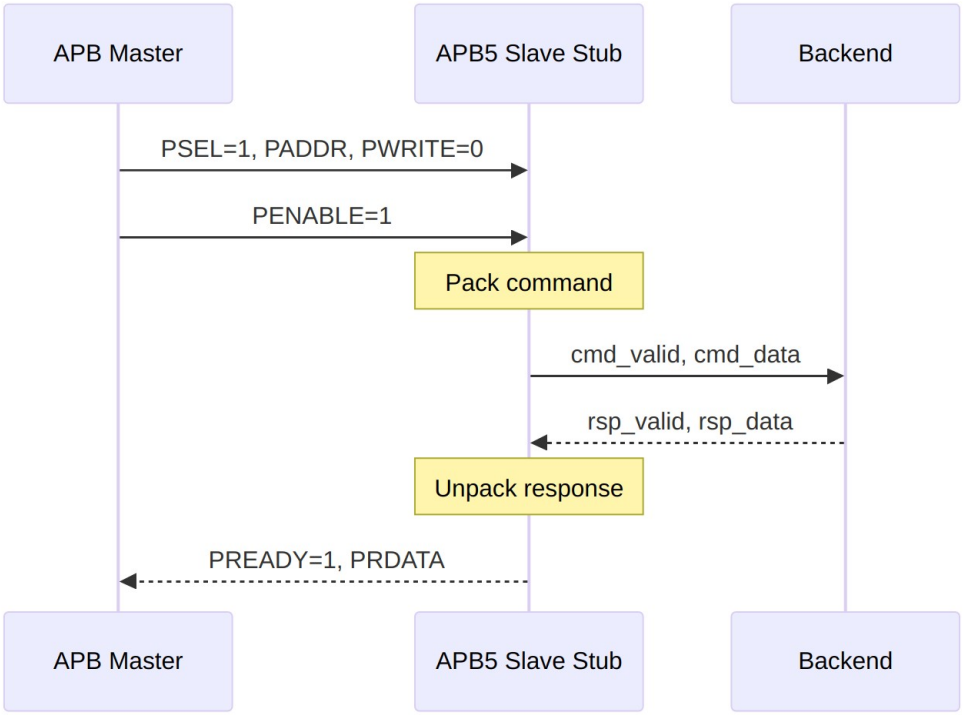
    IDLE --> XFER_DATA : PSEL & PENABLE & cmd_ready
    XFER_DATA --> IDLE : rsp_valid

    state IDLE {
        note right of IDLE : Wait for APB access
    }
    state XFER_DATA {
        note right of XFER_DATA : Wait for backend response
    }
```

10.4.3 State Descriptions

State	Description
IDLE	Waiting for APB transaction (PSEL & PENABLE)
XFER_DATA	Command sent, waiting for response from backend

10.4.4 Transaction Flow



Read Transaction

10.5 Timing Characteristics

Timing diagram pending. The signals and sequence this scenario exercises:

- pclk
- APB signals (PSEL, PENABLE, PADDR, PWRITE, PREADY, PRDATA)
- cmd_valid, cmd_ready, cmd_data
- rsp_valid, rsp_ready, rsp_data
- State transitions IDLE -> XFER_DATA -> IDLE

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

10.6 Usage Examples

```
apb5_slave_stub #(
    .DEPTH          (4),
    .ADDR_WIDTH     (32),
    .DATA_WIDTH     (32),
    .AUSER_WIDTH    (4),
    .WUSER_WIDTH    (4),
    .RUSER_WIDTH    (4),
    .BUSER_WIDTH    (4),
    .ENABLE_PARITY  (0)
) u_apb5_slave_stub (
    .pclk           (apb_clk),
    .presetn        (apb_rst_n),

    // APB5 slave interface
    .s_apb_PSEL     (s_apb_psel),
    .s_apb_PENABLE  (s_apb_penable),
    .s_apb_PADDR    (s_apb_paddr),
    .s_apb_PWRITE   (s_apb_pwrite),
    .s_apb_PWDATA   (s_apb_pwdata),
    .s_apb_PSTRB    (s_apb_pstrb),
    .s_apb_PPROT    (s_apb_pprot),
    .s_apb_PAUSER   (s_apb_pauser),
    .s_apb_PWUSER   (s_apb_pwuser),
    .s_apb_PRDATA   (s_apb_prdata),
    .s_apb_PSLVERR  (s_apb_pslverr),
    .s_apb_PREADY   (s_apb_pready),
    .s_apb_PWAKEUP  (s_apb_pwakeup),   // input: from the Requester
    .s_apb_PRUSER   (s_apb_pruser),
    .s_apb_PBUSER   (s_apb_pbuser),

    // Packed command interface
    .cmd_valid      (be_cmd_valid),
    .cmd_ready      (be_cmd_ready),
    .cmd_data       (be_cmd_data),

    // Packed response interface
    .rsp_valid      (be_rsp_valid),
    .rsp_ready      (be_rsp_ready),
    .rsp_data       (be_rsp_data),
```

```
    // Parity interface (unused here because ENABLE_PARITY=0)
    .s_apb_PADDRCHK      ('0),
    .s_apb_PCTRLCHK      (1'b0),
    .s_apb_PSELCHK       (1'b0),
    .s_apb_PENABLECHK    (1'b0),
    .s_apb_PWDATACHK     ('0),
    .s_apb_PSTRBCHK      (1'b0),
    .s_apb_PAUSERCHK     ('0),
    .s_apb_PWUSERCHK     ('0),
    .s_apb_PRDATACHK     (),
    .s_apb_PREADYCHK     (),
    .s_apb_PSLVERRCHK    (),
    .s_apb_PWAKEUPCHK    (1'b0),
    .s_apb_PRUSERCHK     (),
    .s_apb_PBUSERCHK     (),
    .parity_error_wdata  (),
    .parity_error_ctrl   ()
);

// Simple backend: return stored_data with no error and zeroed user
fields.
// Build the response from the parameters, not from literal widths --
the user
// fields are RUSER_WIDTH and BUSER_WIDTH wide, not a fixed 8 bits.
localparam int RUW = 4;   // RUSER_WIDTH
localparam int BUW = 4;   // BUSER_WIDTH

always_ff @(posedge apb_clk) begin
    if (!apb_rst_n) begin
        be_rsp_valid <= 1'b0;
    end else if (be_cmd_valid && be_cmd_ready) begin
        be_rsp_valid <= 1'b1;
        // {pslverr, prdata, pruser, pbuser}
        be_rsp_data  <= {1'b0, stored_data, {RUW{1'b0}}, {BUW{1'b0}}};
    end else if (be_rsp_ready) begin
        be_rsp_valid <= 1'b0;
    end
end
```

10.7 Design Notes

10.7.1 Skid Buffers

The stub uses skid buffers on both command and response paths: - **Command skid buffer:** Absorbs command while FSM transitions - **Response skid buffer:** Allows backend to push response before slave ready

10.7.2 Parity Implementation

The stub carries its own copy of the `apb5_slave` parity logic, with identical behaviour; the full rules and the per-signal Check Enable terms are in apb5_slave. In short, when `ENABLE_PARITY=1`: - **Odd parity** (IHI 0024E §5.3) on spec-named `*CHK` signals, one check bit per 8 payload bits, computed by per-byte `dataint_parity` instances. - **Checked:** `PADDRCHK`, `PCTRLCHK` (PPROT, PWRITE), `PSELCHK`, `PENABLECHK`, `PWDATACHK`, `PSTRBCHK`, `PAUSERCHK`, `PWUSERCHK`, `PWAKEUPCHK`. - **Generated:** `PRDATACHK`, `PREADYCHK`, `PSLVERRCHK`, `PRUSERCHK`, `PBUSERCHK`. - `parity_error_wdata` (write-data group, while `PSEL & PWRITE`) and `parity_error_ctrl` (address/control group while `PSEL`, `PSELCHK` and `PWAKEUPCHK` at any time out of reset) are combinational and hard-tied to 0 when `ENABLE_PARITY=0`.

10.7.3 Wake-up Handling

`s_apb_PWAKEUP` is an input from the Requester (IHI 0024E §3.7.2, Table 3-7). The stub does not wait for it before PREADY and uses it only for the `PWAKEUPCHK` check; there is no backend `wakeup_request` port.

10.8 Related Modules

- **APB5 Slave** - Full slave module
- **APB5 Master Stub** - Corresponding master stub

10.9 Testing

`val/amba/test_apb5_slave_stub.py` exercises this module. It collects 4 parameter cases at the default `REG_LEVEL`; the two `ENABLE_PARITY=1` cases also run `cocotb_test_apb5_slave_stub_parity`, a directed IHI 0024E ch. 5 check.

```
source env_python
pytest val/amba/test_apb5_slave_stub.py -v
```

10.10 References

- Arm. *AMBA APB Protocol Specification*, ARM IHI 0024E, Feb. 2023 (APB5 first published in Issue D, Apr. 2021). developer.arm.com/documentation/ihi0024/e – ch. 3 “Transfers”, ch. 4 “Operating States”, §3.7 “Wake-up signaling” and §3.8 “User signaling”: the protocol of the wrapped APB5 block; ch. 5 “Interface parity protection” (§5.3, Table 5-1) for the `*CHK` signals. Citation only.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – the valid/ready channel discipline behind the `gaxi_skid_buffer` queues: a buffered stage may stall or add latency without changing what the endpoints compute.
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`)

10.11 Navigation

- **← Back to APB5 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**